



МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное
учреждение высшего образования
«Национальный исследовательский университет «МЭИ»

Институт ИВТИ
Кафедра ВМСС

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(бакалаврская работа)

Направление 09.03.01 Информатика и вычислительная техника
(код и наименование)

Направленность (профиль) Вычислительные машины, комплексы,
системы и сети

Форма обучения очная
(очная/очно-заочная/заочная)

Тема: Разработка клиент-серверного приложения для планирования
и организации мероприятий

Студент А-08-21 Муромцев С.А.
группа подпись фамилия и инициалы

Научный К.Т.Н. доцент Раскатова М.В.
руководитель уч. степень должность подпись фамилия и инициалы

Консультант уч. степень должность подпись фамилия и инициалы

Консультант уч. степень должность подпись фамилия и инициалы

«Работа допущена к защите»

Зав. кафедрой К.Т.Н. доцент Вишняков С.В.
уч. степень звание подпись фамилия и инициалы

Дата _____

Москва, 2025

АННОТАЦИЯ

Данная выпускная работа посвящена разработке клиент-серверного приложения для планирования мероприятий.

Благодаря данному приложению пользователи могут планировать мероприятия, отслеживать их, приглашать на них других пользователей, получать уведомления.

В работе рассмотрены основные технологии разработки веб-приложений, обозначены требования к разработке, спроектирована архитектура и описаны этапы реализации и ключевые особенности.

В состав работы входит 19 рисунков, 7 таблиц и 4 приложения.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	8
1.1. Требования к надёжности и эффективности клиент-серверных приложений.....	8
1.2. Обзор существующих решений: преимущества и недостатки	10
1.3. Архитектура приложений для планирования мероприятий	14
1.3.1. Архитектурные шаблоны	16
1.3.2. Используемые технологии и инструменты	22
2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ПРИЛОЖЕНИЯ.....	29
2.1. Функциональные и нефункциональные требования	29
2.2. Выбранные технологии и инструменты, их преимущества	32
2.3. Проектирование общей архитектуры.....	40
2.3.1. Разработка архитектуры клиентской части приложения	42
2.3.2. Разработка архитектуры серверной части	42
2.3.3. Проектирование структуры базы данных.....	48
3. РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ	53
3.1. Регистрация пользователей	54
3.2. Авторизация пользователей	55
3.3. Управление мероприятиями.....	56
3.4. Календарь мероприятий.....	59
3.5. Тестирование программы	63
ЗАКЛЮЧЕНИЕ	65
СПИСОК ЛИТЕРАТУРЫ.....	68
ПРИЛОЖЕНИЕ А	69
ПРИЛОЖЕНИЕ Б.....	72
ПРИЛОЖЕНИЕ В	76
ПРИЛОЖЕНИЕ Г.....	94

ВВЕДЕНИЕ

В современном мире организация и планирование мероприятий стали неотъемлемой частью как личной, так и профессиональной жизни. С развитием цифровых технологий возрастает потребность в удобных, доступных и эффективных системах для координации событий, управления участниками и оптимизации рабочего процесса. Особенно важную роль системы организации и планирования мероприятий играют в бизнесе, образовании, культуре и общественной деятельности, где своевременное информирование и четкая организация являются залогом успешного проведения мероприятий.

Применение клиент-серверных приложений для планирования и организации мероприятий в бизнесе предоставляет ряд важных преимуществ. Прежде всего, такие приложения позволяют централизовать всю информацию о мероприятиях, обеспечивая быстрый и удобный доступ к данным для всех участников процесса. Автоматизация задач, таких как регистрация участников, бронирование ресурсов, рассылка уведомлений и контроль за выполнением задач, значительно упрощает работу организаторов и снижает вероятность ошибок. Клиент-серверные приложения способствуют ускорению коммуникации внутри команды и с участниками мероприятий, обеспечивая оперативное распространение информации и своевременное реагирование на изменения. Благодаря поддержке различных форматов, офлайн, онлайн и гибридных мероприятий, компании могут гибко адаптироваться к современным условиям. Системы предоставляют инструменты аналитики и отчетности, что позволяет оценивать эффективность мероприятий и выявлять направления для их улучшения. В результате применения таких решений компании экономят время и ресурсы, повышают качество организации мероприятий и обеспечивают безопасность обработки персональных данных участников.

Актуальность данной работы обусловлена тем, что требования к системам планирования и организации мероприятий растут. В современных

условиях бизнеса, где скорость принятия решений, эффективность работы и уровень взаимодействия с клиентами и партнерами играют ключевую роль, организация мероприятий становится важной частью корпоративной культуры и бизнес-процессов. Мероприятия, такие как деловые встречи, конференции, тренинги, презентации и выставки, не только повышают имидж компании, но и играют важную роль в достижении стратегических целей организации. С этим связано возрастание сложности их планирования, координации и выполнения. В связи с этим актуальность разработки клиент-серверных приложений для планирования и организации мероприятий растет с каждым годом. Такие системы могут существенно повысить эффективность организации мероприятий, снизить операционные затраты и улучшить взаимодействие участников.

Целью данной работы является разработка клиент-серверного приложения, предназначенного для эффективного планирования и организации мероприятий, обеспечивающего удобное взаимодействие пользователей, управление событиями и автоматизацию процесса оповещения участников с использованием современных технологий, обеспечивающих возможность последующего расширения функционала.

Для достижения поставленной цели были выделены следующие задачи:

1. Проведение анализа существующих решений в области планирования мероприятий и выявление их основных недостатков и преимуществ.
2. Определение функциональных и нефункциональных требований к разрабатываемому приложению.
3. Проектирование архитектуры системы на основе клиент-серверной модели.
4. Разработка серверной части приложения для обработки данных о пользователях, мероприятиях и уведомлениях.
5. Разработка клиентской части приложения для создания, просмотра и управления мероприятиями.

6. Проектирование структуры базы данных и взаимодействие с хранилищем.

7. Проведение тестирования разработанного приложения с целью проверки его работоспособности и соответствия требованиям.

8. Оценка результатов работы и определение направления для дальнейшего развития системы.

В процессе разработки приложения я освоил принципы проектирования и реализации клиент-серверных систем, предназначенных для планирования, организации и управления мероприятиями. В частности, были изучены архитектурные особенности подобных решений, подходы к эффективной обработке данных, а также современные технологии и инструменты разработки. С учетом специфики проекта, его технических требований и поставленных задач были выбраны наиболее подходящие языки программирования, инфраструктуры, библиотеки, системы управления базами данных и средства тестирования.

В ходе разработки клиент-серверного приложения для планирования и организации мероприятий использовались следующие методы:

- аналитический метод — для изучения существующих решений в области организации мероприятий и выявления их особенностей;
- методы объектно-ориентированного проектирования — для построения архитектуры приложения, моделирования данных и разработки компонентов системы;
- метод системного анализа — для определения требований к функциональности, надежности и безопасности разрабатываемого программного обеспечения;
- методы прототипирования — для создания пользовательского интерфейса и проверки основных сценариев взаимодействия на ранних этапах разработки;
- методы тестирования приложения — для проверки корректности работы системы, выявления и устранения ошибок.

При разработке клиент-серверных приложений целевая аудитория определяется спецификой проекта. Разработанное мной приложение предназначено для взаимодействия между сотрудниками внутри компании и направлено на улучшение их пользовательского опыта за счёт интуитивно понятного интерфейса и высокой скорости отклика системы. С технической стороны решение автоматизирует рутинные задачи, ранее выполнявшиеся вручную, повышает результативность взаимодействия между отделами и обеспечивает быстрый и надёжный обмен информацией внутри организации.

Работа состоит из введения, трех глав, заключения и списка литературы.

Задание на выпускную квалификационную работу приведено в Приложении А.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Требования к надёжности и эффективности клиент-серверных приложений

Для надёжной, безопасной и эффективной работы клиент-серверного приложения, разработанного для планирования и управления мероприятиями, важно соблюдение ряда ключевых условий, которые обеспечивают стабильность системы, защиту данных и удобство взаимодействия пользователей с функционалом сервиса, таких как:

1. Стабильность и отказоустойчивость - система должна обеспечивать непрерывную работу при высоких нагрузках, возникающих, например, в процессе массовой регистрации участников или во время проведения мероприятий. В случае сбоев серверной части приложение должно сохранять работоспособность за счёт механизмов резервирования и восстановления.

2. Производительность в режиме реального времени - приложение должно оперативно обрабатывать пользовательские действия: регистрацию, редактирование событий, обновление расписания, отправку уведомлений и другие критически важные процессы без задержек.

3. Гибкая система ролей и доступов - для обеспечения безопасной работы приложения необходимо реализовать разграничение прав доступа для различных категорий пользователей: организаторов, участников и администраторов.

4. Надёжная регистрация и учёт участников - требуется устойчивый и проверенный механизм регистрации, включающий подтверждение участия и предотвращение дублирования данных. Система должна отслеживать статус каждого участника и обеспечивать достоверность регистрационной информации.

5. Интерактивное и фильтруемое расписание - приложение должно предоставлять пользователям удобный способ просмотра и поиска

мероприятий с возможностью фильтрации по дате, месту проведения, тематике и другим параметрам.

6. Уведомления и оповещения - необходимо реализовать систему уведомлений, информирующую участников о предстоящих мероприятиях, изменениях в расписании, отменах и других важных событиях. Уведомления могут быть реализованы как внутри приложения, так и через внешние каналы (e-mail, push-уведомления и др.).

7. Интеграция с календарями и почтовыми сервисами - поддержка экспорта событий в сторонние календари, например, Яндекс Календарь, Outlook, а также возможность отправки писем с подтверждениями, напоминаниями и информацией об изменениях.

8. Безопасность данных пользователей - приложение должно обеспечивать защиту персональных данных пользователей путём использования современных механизмов шифрования, безопасной авторизации и хранения информации в соответствии с актуальными стандартами.

9. Масштабируемость - архитектура приложения должна предусматривать возможность масштабирования, обеспечивая стабильную работу при увеличении количества пользователей, мероприятий и объёма обрабатываемых данных.

10. Поддержка аналитики и отчётности - необходимо внедрение модулей сбора статистики, анализа посещаемости и активности пользователей, формирования отчётов об эффективности мероприятий и общем уровне взаимодействия с системой.

Клиент-серверное приложение, предназначенное для планирования и организации мероприятий, должно сочетать в себе высокую функциональность, надёжность и удобство использования. Оно должно охватывать весь жизненный цикл мероприятия от регистрации и администрирования до оповещения участников. Успешная реализация такого проекта требует грамотного подбора технологического стека,

ориентированного на стабильную работу, защиту данных и комфортное взаимодействие с пользователем на всех уровнях системы.

1.2. Обзор существующих решений: преимущества и недостатки

Современные клиент-серверные приложения для планирования и организации мероприятий предлагают широкий спектр функций, таких как управление регистрацией, автоматизация процессов, интеграция с платежными системами и аналитика. Однако они также имеют свои ограничения, такие как высокая стоимость, сложности в кастомизации и зависимость от облачных технологий. Я проанализировал некоторые существующие клиент-серверные приложения в области планирования и организации мероприятий и выявил их основные преимущества и недостатки.

Cvent — это облачная платформа для управления мероприятиями, которая помогает компаниям организовывать, планировать и анализировать события.

Преимущества Cvent:

1. Полный набор инструментов — Cvent охватывает весь процесс управления мероприятием: от регистрации и продажи билетов до анализа и отчетности.
2. Масштабируемость — подходит для мероприятий любого масштаба, от небольших встреч до крупных международных конференций.
3. Гибкость в настройках — позволяет адаптировать платформу под специфические нужды каждого мероприятия.
4. Интеграция с другими системами — широкие возможности интеграции с CRM, маркетинговыми платформами и другими корпоративными решениями.
5. Мобильное приложение — персонализированное приложение для участников с возможностью взаимодействия и получения актуальной информации.

6. Обширная аналитика — предоставление детализированных отчетов и данных для улучшения планирования будущих мероприятий.

Недостатки Cvent:

1. Высокая стоимость — Cvent может быть дорогим для малых и средних компаний, особенно с учетом платных дополнительных функций.

2. Сложность в использовании — для новых пользователей интерфейс может быть не таким интуитивно понятным, что требует времени на обучение.

3. Ограниченные возможности кастомизации — в некоторых случаях платформа может быть ограничена в плане персонализации и настроек под уникальные потребности компании.

4. Проблемы с мобильным приложением — хотя платформа предлагает мобильное приложение, оно иногда может быть неудобным или не столь функциональным, как хотелось бы.

5. Зависимость от интернет-соединения — поскольку Cvent — это облачная платформа, стабильное интернет-соединение необходимо для работы, что может быть проблемой в определенных условиях.

Вывод: **Cvent** – это хорошее решение для тех компаний, которые проводят сложные, масштабные мероприятия и хотят управлять всем процессом в одном месте с максимальной автоматизацией и глубокой аналитикой, но для внутренних мероприятий внутри коммерческой организации часто бывает избыточным и слишком дорогим.

Aventri - это платформа для управления мероприятиями (онлайн, офлайн и гибридными), ориентированная в первую очередь на коммерческие организации и корпорации.

Преимущества Aventri:

1. Интуитивно понятный интерфейс — позволяет пользователям легко создавать и управлять мероприятиями, даже без технических навыков.

2. Гибкость и масштабируемость — поддерживает мероприятия любого масштаба, от небольших встреч до крупных конференций.

3. Мощные аналитические инструменты — предлагает детализированные отчёты и аналитику для оценки эффективности мероприятий.

4. Интеграция с другими системами — легко интегрируется с CRM, платежными системами и маркетинговыми платформами.

5. Мобильное приложение — предоставляет персонализированное приложение для участников, улучшая их вовлечённость и взаимодействие с мероприятием.

Недостатки Aventri:

1. Высокая стоимость — может быть дорогим вариантом для малых и средних компаний, особенно с учётом дополнительных функций.

2. Ограниченная кастомизация — возможности для глубокой настройки интерфейса и функций могут быть ограничены.

3. Сложность для новичков — интерфейс и функциональность могут потребовать времени на обучение для пользователей без опыта работы с платформами для мероприятий.

4. Проблемы с производительностью — иногда может наблюдаться медленная работа при большом объёме данных или высокой нагрузке.

5. Ограниченные функции для гибридных мероприятий — в сравнении с конкурентами, платформа может не иметь всех необходимых функций для организации гибридных мероприятий.

Вывод: **Aventri** — это мощный инструмент для компаний, которым нужно проводить профессиональные мероприятия разных форматов: от конференций до гибридных форумов, его использование актуально для больших компаний, агентств, корпораций с регулярными сложными событиями.

Bitrix24 — это универсальная платформа для бизнеса, которая объединяет в себе CRM (систему для управления клиентами), управление задачами и проектами, корпоративный портал для общения сотрудников,

инструменты для планирования, работы с документами, автоматизации бизнес-процессов.

Преимущества Bitrix24:

1. Заменяет сразу несколько сервисов: CRM, управление проектами, задачи, чаты, видеозвонки, календари, документы - всё собрано в одной платформе.
2. Гибкость и настройка — позволяет адаптировать интерфейс и бизнес-процессы под специфические нужды компании.
3. Мобильное приложение — доступность полного функционала на мобильных устройствах для работы в любом месте.
4. Интеграция с внешними сервисами — поддержка множества внешних интеграций (Google, Office 365, социальные сети и т. д.).
5. Бесплатная версия — наличие бесплатного функционала, подходящего для малых и средних компаний.
6. Удобство для команд — встроенные инструменты для чатов, видеоконференций и совместной работы над документами.

Недостатки Bitrix24:

1. Сложность интерфейса — из-за широкого функционала новички могут столкнуться с трудностями в освоении интерфейса.
2. Перегрузка функционала — большое количество опций и настроек может быть избыточным для небольших компаний с простыми задачами.
3. Проблемы с производительностью — при большом объёме данных или пользователей система может работать медленно.
4. Ограниченная поддержка в бесплатной версии — бесплатная версия имеет ограничения по функционалу и числу пользователей.
5. Не всегда интуитивно понятные бизнес-процессы — для некоторых компаний настройка автоматизации и процессов может быть сложной.

Вывод: **Bitrix24** подходит для компаний любого размера, особенно тем, кто хочет объединить всё в одной системе и сэкономить на покупке множества отдельных сервисов.

1.3. Архитектура приложений для планирования мероприятий

Архитектура клиент-серверных приложений представляет собой модель организации программной системы, при которой функции распределяются между двумя основными компонентами клиентом и сервером. Эта архитектура основывается на сетевом взаимодействии, при котором клиент отправляет запросы на выполнение определённых операций, а сервер обрабатывает эти запросы, выполняет необходимую бизнес-логику и возвращает результат клиенту.

Клиентская часть приложения представляет собой компонент, отвечающий за взаимодействие с пользователем. Она обеспечивает отображение интерфейса, отправку запросов к серверу и обработку ответов. В зависимости от типа платформы клиент может быть реализован в виде веб-приложения, мобильного приложения или десктопной программы.

Основными задачами клиентской части являются формирование и отображение пользовательского интерфейса (например, календаря или списка мероприятий), предоставление функциональности для создания, редактирования и просмотра событий, а также организация сетевого взаимодействия с серверной частью приложения. Помимо технической составляющей, важной задачей клиента является обеспечение удобства и интуитивной понятности интерфейса, что напрямую влияет на качество пользовательского опыта.

Примеры технологий: React, Flutter, Android (Kotlin), iOS (Swift).

Серверная часть представляет собой центральный компонент программной системы, отвечающий за выполнение бизнес-логики, обработку данных и обеспечение взаимодействия с хранилищами информации и внешними сервисами. Основной задачей сервера является приём и обработка

запросов, поступающих от клиентских приложений, с последующим выполнением необходимой бизнес-логики, включая проверку прав доступа, применение правил обработки событий и управление данными.

Сервер осуществляет постоянное взаимодействие с базой данных, где хранятся все ключевые элементы системы, такие как сведения о пользователях, мероприятиях, приглашениях и напоминаниях. Кроме того, сервер может выполнять интеграцию с внешними сервисами: системами отправки уведомлений, например, по электронной почте или через push-уведомления, очередями задач для реализации отложенной обработки, а также сторонними API, такими как календарные сервисы, например, Яндекс Календарь, что расширяет функциональные возможности приложения и обеспечивает его интеграцию в более широкий цифровой контекст.

Примеры технологий: Node.js, Django, Spring Boot, FastAPI.

База данных представляет собой ключевой компонент программной системы, предназначенный для надёжного хранения, структурирования и управления всей информацией, необходимой для полноценного функционирования приложения. В рамках клиент-серверной архитектуры база данных выполняет роль централизованного хранилища, с которым взаимодействует серверная часть, осуществляя чтение, запись и модификацию данных в ответ на запросы от клиентской части.

Хранимая информация охватывает широкий спектр сущностей, включая учетные записи пользователей, сведения о мероприятиях, приглашениях, участниках, а также напоминания и уведомления. Эти данные являются основой для реализации основной функциональности системы, связанной с планированием и организацией событий. Таким образом, база данных обеспечивает целостность и согласованность информации, а также её доступность для всех компонентов приложения в процессе обработки запросов и выполнения бизнес-логики.

Примеры: PostgreSQL, MySQL, MongoDB.

Целостность и надёжность функционирования базы данных обеспечиваются соблюдением ACID-свойств, которые представляют собой совокупность четырёх ключевых принципов: атомарности, согласованности, изолированности и долговечности. Эти свойства являются основой надёжной обработки транзакций в системах управления базами данных.

Атомарность (Atomicity) означает, что каждая транзакция выполняется как единое целое. Либо все её операции выполняются успешно, либо, в случае ошибки, все изменения отменяются, и база данных остаётся в исходном состоянии.

Согласованность (Consistency) гарантирует, что после выполнения транзакции база данных остаётся в корректном состоянии, соответствующем всем установленным ограничениям, правилам и логике предметной области. Каждая транзакция переводит базу из одного согласованного состояния в другое.

Изолированность (Isolation) означает, что параллельно выполняющиеся транзакции не должны влиять друг на друга. Изменения, внесённые одной транзакцией, становятся видимыми для других только после её завершения, что позволяет избежать некорректного чтения или обновления данных при одновременном доступе.

Долговечность (Durability) обеспечивает сохранность всех изменений, внесённых в результате успешно завершённой транзакции, даже в случае сбоев оборудования или программного обеспечения. Это достигается путём надёжной фиксации данных на постоянных носителях.

Соблюдение ACID-свойств критически важно для обеспечения корректной и предсказуемой работы базы данных, особенно в условиях многопользовательского доступа и высокой нагрузки.

1.3.1. Архитектурные шаблоны

При разработке клиент-серверных приложений применяются различные архитектурные шаблоны, каждый из которых решает конкретные

задачи — от организации кода до масштабируемости и безопасности. Вот основные шаблоны, которые часто используются:

1. MVC (Model–View–Controller).

Этот архитектурный подход используется для разделения логики приложения на три основных компонента, что обеспечивает чистую архитектуру, облегчает сопровождение и тестирование системы.

Клиентская сторона (React):

- **View (Представление)** — компоненты пользовательского интерфейса (например, страницы и формы) отвечают за отображение данных, которые поступают из модели, и за визуальное взаимодействие с пользователем.

- **Controller (Контроллер)** — обработчики событий и логика управления, которая управляет взаимодействием между моделью и представлением, отвечает за обработку действий пользователя, таких как клики на кнопки или отправка форм.

- **Model (Модель)** — состояние приложения, которое хранит данные и логику их изменения. Например, состояние компонента, управляемое с помощью хуков `useState` или глобального состояния, управляемого с помощью `Redux`, выступает в роли модели, отвечающей за хранение и обновление данных.

Серверная сторона (Spring Boot):

- **Controller (Контроллер)** — REST API контроллеры, реализующие обработку HTTP-запросов. Они отвечают за принятие запросов от клиента и возврат соответствующих ответов. Например, контроллеры могут обрабатывать запросы на получение, создание, изменение и удаление данных.

- **Model (Модель)** — сущности и DTO (Data Transfer Object) классы, представляющие структуру данных и бизнес-логику на серверной стороне. Эти классы используются для работы с базой данных через JPA и передачи данных между слоями приложения.

- **View (Представление)** — в случае использования шаблонов серверной генерации, данные преобразуются в представление с использованием шаблонизаторов, таких как **Thymeleaf**. Он позволяет генерировать HTML-страницы на сервере, которые отправляются на клиент. В другом случае, при взаимодействии через API, данные возвращаются в формате JSON, который служит представлением данных для клиента.

Такое разделение обеспечивает чёткое разграничение обязанностей и облегчает поддержку, масштабирование и тестирование как клиентской, так и серверной частей приложения.

2. **Client–Server (Клиент–Сервер).**

При данной клиент-серверной архитектуре приложение разделено на две основные части:

1) **клиентская часть** — это **веб-приложение**, построенное с использованием **React[5]**, которое обеспечивает **пользовательский интерфейс (UI)**. Клиентская часть отвечает за взаимодействие с пользователем, отображение данных, обработку пользовательских действий (например, кликов и ввода данных) и отправку запросов на сервер;

2) **серверная часть** — реализована с использованием **Spring Boot[1]**, который обрабатывает **бизнес-логику** приложения и управляет взаимодействием с базой данных. Сервер отвечает за обработку запросов от клиента, выполнение операций с данными и отправку ответов.

Клиент-серверное взаимодействие осуществляется через **REST API** по протоколу **HTTP** с использованием формата **JSON** для передачи данных. Все данные, такие как информация о пользователях, событиях и уведомлениях, передаются между клиентом и сервером в виде JSON-объектов, что обеспечивает стандартизированный и эффективный обмен информацией между компонентами приложения.

Таким образом, архитектура приложения позволяет разделить ответственность за различные аспекты работы системы, улучшая её масштабируемость, поддержку и безопасность.

3. REST (Representational State Transfer).

REST обеспечивает эффективное и стандартизированное взаимодействие между клиентом и сервером через использование стандартных HTTP-методов (GET, POST, PUT, DELETE). Этот архитектурный стиль позволяет упрощённо организовать обмен данными между компонентами приложения, сохраняя при этом гибкость и масштабируемость системы.

Серверная часть приложения реализует **REST-интерфейс**, предоставляя набор **HTTP-эндпоинтов**, через которые осуществляется доступ к различным данным и функционалу системы. Эти эндпоинты обрабатывают запросы от клиента, выполняют необходимые операции и возвращают ответы в формате **JSON**.

- **GET** — используется для получения данных (например, список событий или детали пользователя).
- **POST** — применяется для создания новых записей (например, создание нового события).
- **PUT** — используется для обновления существующих данных (например, изменение информации о событии).
- **DELETE** — служит для удаления данных (например, удаление события).

Клиентская часть, реализованная с использованием **React**, направляет **HTTP-запросы** к этим эндпоинтам, чтобы **получать, изменять или удалять** информацию, тем самым обеспечивая взаимодействие с сервером в рамках архитектуры **REST**.

Такой подход позволяет изолировать логику обработки данных на сервере, а на клиенте сосредоточиться на представлении данных и взаимодействии с пользователем.

4. Single Page Application (SPA).

Основной целью архитектуры **Single Page Application (SPA)** является улучшение пользовательского опыта через динамическую загрузку данных и

контента. В отличие от традиционных приложений, где для каждого взаимодействия с сервером загружается новая страница, SPA загружает весь интерфейс только один раз, а последующие взаимодействия с сервером происходят через API. Это позволяет избежать полной перезагрузки страницы и делает приложение более быстрым и отзывчивым.

Клиентская часть реализуется как **Single Page Application (SPA)** с использованием **React**. Это позволяет динамически обновлять содержимое интерфейса без необходимости перезагрузки страницы. Все данные, необходимые для отображения, запрашиваются с сервера через **REST API**, что обеспечивает плавную работу приложения, без задержек при переходах между различными разделами, быстрое обновление интерфейса за счёт частичной загрузки данных, вместо полной перезагрузки, снижение нагрузки на сервер, так как только необходимые данные отправляются и обрабатываются без перезагрузки всей страницы.

В результате, пользователи получают более интерактивное и быстрое приложение, что способствует улучшению взаимодействия с системой.

5. CORS и безопасность.

Основной целью настройки **CORS (Cross-Origin Resource Sharing)** является обеспечение безопасности взаимодействия между клиентом и сервером, особенно в случаях, когда они находятся на разных доменах. Например, если клиентская часть приложения работает на **localhost:3000**, а серверная — на **localhost:8080**, необходимо обеспечить безопасный обмен данными между ними, предотвратив возможные атаки с других, несанкционированных источников.

CORS-фильтр настраивается на серверной стороне для контроля доступа при выполнении HTTP-запросов с других доменов или портов. Это позволяет:

- ограничить доступ к API-сервисам только для разрешённых источников (например, клиентская часть с **localhost:3000**);

- исключить возможность выполнения несанкционированных запросов с чужих доменов, что помогает предотвратить такие атаки, как **Cross-Site Request Forgery (CSRF)** и **Cross-Site Scripting (XSS)**;

- повысить уровень безопасности приложения, гарантируя, что только клиент, который является доверенным источником, может обращаться к серверным ресурсам.

Таким образом, правильная настройка **CORS** играет важную роль в защите данных и предотвращении угроз, связанных с несанкционированными запросами.

6. Многослойная архитектура.

Многослойная архитектура предполагает разделение приложения на несколько логических слоёв, каждый из которых отвечает за свою специфическую задачу. Это разделение помогает улучшить поддерживаемость, расширяемость и тестируемость системы, а также способствует упрощению разработки и сопровождения. В типичной многослойной архитектуре выделяют три основных слоя: **представление (UI)**, **логика (сервис)** и **доступ к данным (DAO/repository)**.

Ключевые слои архитектуры и их функции:

1. **Контроллеры (Controller)** — обрабатывают входящие **HTTP-запросы** от клиента, выполняют проверку авторизации и прав доступа, а также передают данные в соответствующий слой бизнес-логики. Контроллеры выступают в роли посредников между клиентом и внутренними слоями приложения, управляя потоком данных.

2. **Сервисный слой (Service)** — инкапсулирует всю бизнес-логику приложения. Он отвечает за выполнение различных операций, таких как обработка данных, выполнение вычислений, вызовы других сервисов или интеграция с внешними системами. Сервисный слой обеспечивает разделение ответственности, улучшая читабельность кода и упрощая его тестирование.

3. **Репозитории (Repository)** — осуществляют взаимодействие с базой данных, предоставляя абстракцию для работы с сущностями через **JPA**. Репозитории отвечают за выполнение операций **CRUD** (создание, чтение, обновление, удаление), а также за выполнение сложных запросов к базе данных. Этот слой позволяет отделить логику взаимодействия с данными от бизнес-логики.

Таким образом, многослойная архитектура помогает структурировать приложение таким образом, чтобы каждая его часть отвечала за чётко определённые функции. Это позволяет эффективно разрабатывать и поддерживать приложение, улучшать масштабируемость и производительность, а также упрощать тестирование и сопровождение.

В целом использование перечисленных выше подходов обеспечивает создание стабильного, структурированного и легко масштабируемого веб-приложения, в котором каждый компонент выполняет чётко определённую функцию и взаимодействует с остальными через установленную архитектурную модель.

1.3.2. Используемые технологии и инструменты

Правильный выбор технологий и инструментов является одним из самых важных аспектов, который значительно влияет на успешность разработки клиент-серверного приложения. Он определяет не только эффективность работы системы, но и её стабильность, безопасность, а также возможность масштабирования и интеграции с другими сервисами. Под технологиями и инструментами в данном контексте понимаются языки программирования, фреймворки, библиотеки, базы данных, средства для тестирования и деплоя, а также технологии для обеспечения безопасности, мониторинга производительности и взаимодействия между клиентом и сервером. Выбор этих компонентов должен быть основан на требованиях проекта, предполагаемой нагрузке, типе пользователей и других специфичных факторах.

Языки программирования и технологии, используемые при разработке клиент-серверных приложений.

Для разработки клиент-серверных приложений в области планирования мероприятий используется несколько языков программирования, каждый из которых решает определённые задачи на стороне клиента или сервера. Выбор языка зависит от требований к функционалу, производительности и удобству разработки. Также применяются фреймворки и библиотеки, которые ускоряют создание, тестирование и поддержку приложений, обеспечивая высокое качество кода и гибкость.

Языки программирования и технологии для разработки серверной части.

Серверная часть отвечает за обработку данных, взаимодействие с базой данных и логику приложения. Популярные языки для серверной разработки:

1. Python

Фреймворки: Django, Flask.

Python является простым и мощным языком для быстрого прототипирования и разработки серверной логики. Фреймворки Django и Flask предоставляют богатый функционал для разработки RESTful API, обработки данных и интеграции с базами данных.

2. JavaScript (Node.js)

Фреймворки: Express.js, NestJS.

Node.js позволяет использовать JavaScript для разработки серверной части приложения. Это особенно удобно, если уже используется JavaScript на клиентской стороне. Node.js имеет большое сообщество и позволяет создавать масштабируемые серверы с высокой производительностью.

3. Java[4]

Фреймворки: Spring, Spring Boot, Spring Data, Spring Security.

Java хорошо подходит для крупных и масштабируемых приложений. Фреймворк Spring облегчает создание высоконагруженных серверных решений и интеграцию с различными сервисами.

4. PHP

Фреймворки: Laravel, Symfony.

PHP широко используется для разработки веб-приложений. Laravel и Symfony — это мощные фреймворки, которые предоставляют удобные инструменты для работы с базами данных, аутентификацией и маршрутизацией запросов.

5. C# (ASP.NET Core):

Фреймворки: ASP.NET Core.

C# используется для создания надежных и производительных серверных приложений. ASP.NET Core — это современный фреймворк для создания веб-приложений с поддержкой RESTful API и интеграции с базами данных.

Языки программирования и технологии для разработки клиентской части.

Клиентская часть — это интерфейс, с которым взаимодействует пользователь, и он должен быть удобным, интуитивно понятным и адаптивным. Для разработки клиентской части используются следующие языки и технологии:

1. **HTML/CSS/JavaScript** - основные технологии для создания веб-интерфейса. HTML и CSS отвечают за структуру и стилизацию страницы, а JavaScript — за динамическое взаимодействие с пользователем, например, обработка событий, отправка запросов на сервер.

2. **React.js**[11] - популярная библиотека JavaScript для построения пользовательских интерфейсов. React позволяет создавать компонентные и масштабируемые приложения, что идеально подходит для динамичных интерфейсов планирования мероприятий.

3. **Vue.js** - простой в освоении фреймворк для создания интерактивных веб-интерфейсов. Vue.js подходит для разработки как небольших, так и крупных приложений.

4. **TypeScript** - расширение JavaScript, добавляющее статическую типизацию, что позволяет легче поддерживать и масштабировать код. Часто используется вместе с фреймворком React.

Базы данных, используемые при разработке клиент-серверных приложений.

При разработке клиент-серверных приложений для планирования и организации мероприятий, выбор базы данных является критически важным для обеспечения эффективного хранения, обработки и извлечения данных о мероприятиях, участниках, расписаниях, задачах и других сущностях. База данных должна быть надежной, масштабируемой и обеспечивать удобный доступ к данным.

Различают два основных типа баз данных, которые могут быть использованы при разработке клиент-серверного приложения.

1. Реляционные базы данных (SQL).

Реляционные базы данных являются наиболее популярным выбором для хранения структурированных данных, таких как расписания мероприятий, информация о пользователях, регистрациях и другие данные, которые могут быть представлены в виде таблиц. Они поддерживают сложные запросы и обеспечивают целостность данных[12].

Популярные реляционные базы данных:

1) **PostgreSQL** — это одна из самых мощных и популярных реляционных баз данных с открытым исходным кодом. PostgreSQL поддерживает расширенные функции, такие как транзакции, сохраненные процедуры, поддержку JSON и полнотекстовый поиск. Она идеально подходит для приложений, где требуется высокая надежность и масштабируемость[2];

2) **MySQL/MariaDB**: MySQL — это также одна из самых популярных реляционных баз данных, которая используется для большинства веб-приложений. MariaDB — это форк (ответвление) MySQL, который активно развивается и имеет дополнительные функции и

улучшения. Оба решения хорошо подходят для небольших и средних приложений, включая веб-сайты и системы управления мероприятиями;

3) **Microsoft SQL Server** — это коммерческая реляционная база данных от Microsoft, которая используется в крупных организациях и для корпоративных приложений. SQL Server обладает хорошими инструментами для анализа данных и обработки больших объемов информации.

Преимущества реляционных баз данных заключаются в следующем: они обеспечивают чёткую организацию и структурирование информации, поддерживают мощный язык запросов SQL для выборки и обработки данных, гарантируют целостность и согласованность благодаря транзакциям, а также оптимально подходят для систем с формализованной схемой данных — например, для приложений, связанных с управлением событиями, расписаниями и пользователями.

2. Нереляционные базы данных (NoSQL).

Для приложений, где данные могут быть менее структурированными или могут изменяться со временем, например, данные о мероприятиях, гибкость в хранении дополнительных атрибутов для каждого мероприятия, могут быть использованы NoSQL базы данных. Такие базы данных хорошо подходят для хранения данных, которые могут варьироваться по структуре.

Популярные NoSQL базы данных:

1) **MongoDB** — это документно-ориентированная база данных с открытым исходным кодом, которая хранит данные в формате BSON (JSON-подобные документы). MongoDB позволяет гибко управлять данными, добавлять новые поля и изменять структуру данных без необходимости миграции схемы. Это идеально подходит для хранения информации о мероприятиях, в которой могут быть разные атрибуты для разных типов мероприятий;

2) **CouchDB** — еще одна документно-ориентированная база данных, которая использует JSON для хранения данных. CouchDB обеспечивает

высокую доступность и отказоустойчивость, что полезно для приложений, которые должны быть всегда доступными.

Преимущества NoSQL баз данных включают: гибкость в моделировании данных без необходимости строго заданной схемы, что упрощает адаптацию под изменяющиеся требования; высокую масштабируемость и производительность при работе с большими объёмами информации; возможность хранения вложенных структур, например, документов, что особенно удобно для представления комплексных объектов, таких как события с множеством параметров.

Системы потоковой обработки данных (стриминговые платформы).

В дополнение к традиционным базам данных при разработке клиент-серверных приложений могут использоваться системы потоковой обработки данных, позволяющие обрабатывать и передавать информацию в режиме реального времени. Одной из наиболее широко применяемых платформ в этой категории является **Apache Kafka**[8].

Apache Kafka — это распределённая платформа для публикации, подписки, хранения и обработки потоков данных. Она особенно полезна в ситуациях, когда необходимо мгновенно реагировать на события — например, при регистрации участников мероприятия, обновлении расписания, отправке уведомлений или отслеживании активности пользователей. Kafka реализует концепцию "логов событий", обеспечивая надёжную и масштабируемую передачу информации между различными компонентами приложения.

Kafka не является базой данных в традиционном смысле: она не предназначена для долговременного хранения и запроса данных, как реляционные или NoSQL СУБД. Вместо этого она часто используется в качестве промежуточного слоя между источниками данных (например, пользовательскими действиями или микросервисами) и системами хранения (базами данных, аналитическими хранилищами и т. д.).

Применение Kafka в рамках архитектуры клиент-серверного приложения позволяет:

- обрабатывать события в реальном времени;
- эффективно масштабировать взаимодействие между сервисами;
- повысить надёжность и отказоустойчивость системы;
- реализовать событийно-ориентированную архитектуру, способную
- гибко адаптироваться к росту нагрузки и изменениям бизнес-логики.

Таким образом, Apache Kafka является важным инструментом при построении современных, высоконагруженных и динамичных систем для управления мероприятиями.

Вывод: на начальном этапе работы над проектом мной была проведена всесторонняя аналитическая подготовка. В первую очередь я изучил общие требования к клиент-серверным приложениям, что позволило определить ключевые характеристики, которым должна соответствовать разрабатываемая система. Далее был проведён обзор и анализ существующих решений, их функциональных возможностей, а также преимуществ и недостатков. Особое внимание в процессе проектирования было уделено рассмотрению существующих языков программирования, систем управления базами данных и фреймворков, применяемых для разработки клиентской и серверной части веб-приложений. Также были детально рассмотрены архитектурные подходы, включая шаблоны проектирования, такие как MVC и REST, CORS-фильтр, SPA, многослойная архитектура, что позволило осознанно подойти к выбору наиболее эффективной структуры приложения. Полученные в результате исследования выводы легли в основу проектирования архитектуры разработанной системы планирования и организации мероприятий.

2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ПРИЛОЖЕНИЯ

При разработке клиент-серверного приложения необходимо обеспечить реализацию полного набора операций по управлению событиями — создание, редактирование, удаление и просмотр посредством веб-интерфейса. Архитектура приложения должна предусматривать разделение на клиентскую и серверную части, взаимодействующие между собой через протокол HTTP с использованием REST API. Приложение предполагается развернуть на локальном сервере с обязательной реализацией CORS-фильтра для корректной междоменной передачи данных. Общая схема взаимодействия основных компонентов приведена на рис.1. Более наглядная схема приведена в Приложении Б (рис.Б.1.).

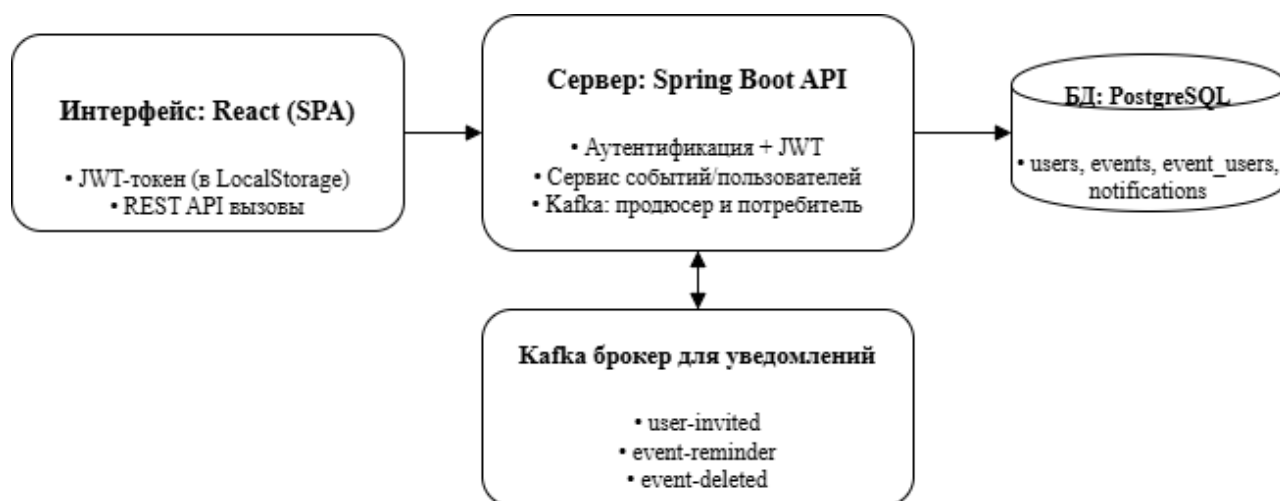


Рис.1. Общая схема приложения

2.1. Функциональные и нефункциональные требования

Функциональные требования к разработке приложения календаря событий:

1. Приложение должно поддерживать возможность регистрации новых пользователей через веб-форму с валидацией введенных данных. Пользователи должны иметь возможность создать учетную запись, указав необходимые данные (имя, фамилию, email и пароль).

2. После регистрации пользователи должны иметь возможность войти в систему с использованием email и пароля. Для авторизации должна использоваться система JWT для создания и проверки токенов.

3. Пользователи должны иметь возможность создавать, редактировать и удалять мероприятия. Каждый пользователь, создающий мероприятие, автоматически становится его организатором, а также должны быть реализованы возможности добавления и удаления участников.

4. Организаторы мероприятий должны иметь возможность изменять место и время проведения мероприятия, а все изменения должны синхронизироваться с сервером и обновляться в базе данных.

5. Приложение должно поддерживать отображение списка предстоящих мероприятий в виде календаря с возможностью фильтрации по дате, статусу и роли пользователя. Пользователи должны иметь возможность просматривать мероприятия, к которым они привязаны, и фильтровать их по различным параметрам.

6. Участники мероприятий должны получать уведомления и напоминания о событиях до их начала, которые отображаются на клиентской стороне, информируя о предстоящих мероприятиях.

7. Все ключевые действия пользователей, такие как регистрация, авторизация, создание, редактирование и удаление мероприятий, должны быть зафиксированы в журнале логирования для дальнейшего анализа и отладки.

8. Все операции, требующие авторизации (например, создание, редактирование и удаление мероприятий), должны быть защищены с использованием JWT и Spring Security. Доступ к мероприятиям должен контролироваться в зависимости от роли пользователя (организатор или участник).

Нефункциональные требования:

1. Производительность:

- приложение должно обеспечивать быстрый отклик на запросы пользователей, включая создание, редактирование и удаление мероприятий;

- платформа должна поддерживать большое количество пользователей и мероприятий без существенного падения производительности;

2. Масштабируемость:

- система должна быть легко масштабируемой, чтобы поддерживать увеличение числа пользователей и объём данных (например, добавление новых серверов или баз данных для обработки нагрузки);

3. Безопасность:

все пароли должны шифроваться с использованием алгоритма BCrypt перед хранением в базе данных;

- JWT должен использоваться для защиты API-эндпоинтов и авторизации запросов;

- логирование должно включать информацию о попытках несанкционированного доступа и других инцидентах безопасности.

4. Удобство использования:

- интерфейс пользователя должен быть интуитивно понятным, с возможностью быстрого добавления, редактирования и удаления мероприятий;

- приложение должно предоставлять удобный интерфейс для просмотра календаря мероприятий и фильтрации по различным параметрам.

5. Надёжность и отказоустойчивость:

- приложение должно быть устойчивым к сбоям, обеспечивая сохранение данных и минимальное время простоя;

- все операции с базой данных должны быть атомарными, и система должна быть способна восстанавливать данные в случае сбоя.

6. Логирование:

- должно обеспечивать подробную информацию для администраторов и разработчиков о всех критических событиях;

2.2. Выбранные технологии и инструменты, их преимущества

С учетом особенностей проекта, технических требований и целей мной были выбраны наиболее подходящие технологии и инструменты:

- **JavaScript** — язык программирования для разработки клиентской части;
- **Java** — объектно-ориентированный язык программирования для создания устойчивого и масштабируемого приложения на серверной стороне;
- **React** — это фреймворк (библиотека) для разработки пользовательского интерфейса;
- **Spring Boot** - фреймворк для Java, предназначенный для упрощения разработки серверных приложений;
- **Spring Data** - фреймворк для работы с базами данных в приложениях на платформе Spring. Он упрощает интеграцию с различными типами баз данных и автоматизирует многие операции, связанные с хранением данных;
- **Spring Security** — инструмент в рамках фреймворка Spring, который предоставляет возможности для аутентификации и авторизации пользователей, а также защиту от различных видов атак (например, CSRF, XSS и т.д.);
- **PostgreSQL** — система управления базами данных (СУБД), которая используется для хранения и обработки данных о мероприятиях, участниках, задачах и ресурсах;
- **Apache Kafka** — распределённая платформа для потоковой передачи и обработки данных в реальном времени, которая используется для организации системы уведомлений. События, такие как приглашение пользователя на мероприятие, напоминание о событии или его удаление, обрабатываются для отправки уведомлений.
- **Docker** — инструмент для контейнеризации приложений и обеспечения их совместимости;

- **JUnit** — инструмент для тестирования серверных приложений на Java;
- **Mockito** — инструмент для автоматизированного тестирования как клиентской, так и серверной части.

Эти технологии были выбраны мной с целью обеспечения надежности и безопасности приложения, а также для оптимизации процесса разработки, поскольку они обладают рядом ключевых преимуществ по сравнению с другими вариантами[9, 10].

JavaScript имеет несколько ключевых преимуществ перед другими языками программирования для разработки клиентской части приложения:

1. Простота разработки и тестирования — JavaScript не требует сложной настройки для работы на локальном сервере. Разработчик может сразу запустить сервер и начать тестировать функциональность, что ускоряет процесс разработки.
2. Динамическое обновление интерфейса — JavaScript позволяет обновлять элементы интерфейса без перезагрузки страницы, что улучшает взаимодействие с пользователем. Например, добавление, редактирование или удаление событий в календаре происходит мгновенно без необходимости обновления всей страницы.
3. Легкость интеграции с REST API — JavaScript, с использованием таких инструментов, как Axios, легко отправляет HTTP-запросы на локальный сервер и обрабатывает ответы, что позволяет эффективно взаимодействовать с бэкендом для получения или отправки данных о событиях.
4. Поддержка асинхронности — JavaScript отлично справляется с асинхронными операциями, такими как получение данных о событиях с сервера или отправка новых данных без блокировки интерфейса.
5. Снижение нагрузки на сервер — за счет работы на локальном хосте клиентская часть может взаимодействовать с сервером минимальное

количество раз (например, при каждом запросе данных или отправке формы), что делает архитектуру более легкой и эффективной в условиях разработки.

6. Обширная экосистема библиотек — JavaScript предоставляет огромное количество библиотек для реализации календарей, форм и интерфейсов, что значительно ускоряет разработку и упрощает интеграцию с сервером.

7. Быстрая реакция на действия пользователя — благодаря JavaScript интерфейс приложения становится интерактивным и отзывчивым, что особенно важно для таких приложений, как календарь, где важна скорость обновления данных и удобство пользования.

8. Совместимость с браузерами — JavaScript гарантирует, что приложение будет работать на большинстве современных браузеров, что позволяет пользователям легко использовать приложение без необходимости устанавливать дополнительные плагины.

Эти особенности делают JavaScript незаменимым инструментом для создания веб-приложения, предоставляя ему явные преимущества перед другими языками при разработке фронтенда.

Java обладает рядом преимуществ для разработки серверной части клиент-серверного приложения для планирования мероприятий по сравнению с другими языками программирования:

1. Мощная экосистема и поддержка библиотек — Java имеет богатую экосистему библиотек и фреймворков, например, Spring Boot, Spring Data, Spring Security, которые упрощают создание и поддержку серверной части. Это ускоряет процесс разработки и внедрения новых функций, таких как обработка запросов, работа с базой данных, реализация CORS-фильтра и MVC-архитектуры, а также позволяют быстро разрабатывать RESTful API для взаимодействия с клиентской частью.

2. CORS-фильтр для безопасности — Java предоставляет мощные механизмы для реализации CORS (Cross-Origin Resource Sharing) фильтров, что позволяет контролировать доступ к серверу и предотвращать

нежелательные запросы из других источников. Это особенно важно для приложений, работающих в браузере, чтобы защитить сервер от несанкционированного доступа.

3. Сильная типизация — Java известна своей строгой типизацией, что помогает избежать многих ошибок на этапе компиляции. Это позволяет гарантировать высокую стабильность серверной части и улучшить читаемость и поддержку кода.

4. Поддержка многозадачности и масштабируемости — Java обеспечивает эффективную работу с многозадачностью, позволяя серверам обрабатывать несколько запросов одновременно без значительных потерь производительности. Это особенно важно для серверной части, которая будет обрабатывать запросы на создание, редактирование и удаление событий.

5. Сильная интеграция с базами данных — Java и такие фреймворки, как Spring Data, позволяют эффективно работать с реляционной базой данных PostgreSQL, что идеально подходит для хранения информации о событиях и пользователях. Эти технологии предоставляют удобные методы для выполнения CRUD-операций и сложных запросов.

6. Гибкость в реализации MVC-архитектуры — Java с использованием фреймворков, отлично поддерживает архитектуру MVC (Model-View-Controller), что позволяет разделить логику обработки запросов, бизнес-логику и представление данных. Это упрощает поддержку и расширение приложения, улучшая его структуру и читабельность кода.

7. Удобство для тестирования и автоматизации — Java обладает богатым набором инструментов для модульного и интеграционного тестирования, например, JUnit, Mockito. Это позволяет легко проводить автоматизированные тесты для проверки правильности работы серверной логики, что повышает надежность приложения.

8. Производительность и надежность — Java обеспечивает высокую производительность и стабильность, что делает её идеальным выбором для

серверной части приложения, которое должно работать быстро и без сбоев, обеспечивая бесперебойную работу календаря событий.

9. Сообщество и документация — Java имеет огромное и активное сообщество, что облегчает решение любых проблем, связанных с разработкой. Хорошо документированные фреймворки и инструменты позволяют быстро разобраться в тонкостях работы с серверной частью и обеспечить высокое качество кода.

10. Интеграция с другими системами и сервисами - Java широко используется для интеграции с различными системами и сервисами через API, что важно для приложения для планирования мероприятий. Например, можно интегрировать систему с внешними календарями (Яндекс Календарь, Outlook), социальными сетями для регистрации пользователей.

11. Поддержка микросервисной архитектуры - Java идеально подходит для разработки микросервисных приложений, что позволяет разрабатывать серверную часть в виде независимых, легко масштабируемых сервисов. Это важно для гибкости и масштабируемости приложения для планирования мероприятий, когда требуется адаптация под изменяющиеся потребности бизнеса.

Таким образом, использование Java для разработки серверной части клиент-серверного приложения позволяет добиться высокой безопасности, производительности, масштабируемости и удобства разработки, особенно при реализации архитектуры MVC с использованием CORS-фильтра.

Использование фреймворка **React** при разработке клиентской части клиент-серверного приложения на локальном сервере имеет свои преимущества:

1. Компонентный подход — React позволяет разрабатывать интерфейс как набор независимых переиспользуемых компонентов (например, календарь, форма события, список событий), что упрощает сопровождение и масштабирование приложения.

2. Виртуальный DOM — благодаря виртуальному DOM React обновляет только изменённые части интерфейса, что обеспечивает высокую производительность даже при частых изменениях данных (например, при переключении дат или обновлении событий).

3. Односторонняя передача данных (data flow) — упрощает отладку и контроль состояния приложения, особенно при работе с REST API, когда данные приходят с сервера и отображаются в компонентах.

4. Развитая экосистема — наличие множества готовых библиотек (React Router, Axios и др.) позволяет быстро реализовать маршрутизацию, сетевые запросы, формы и валидацию без лишнего кода.

5. Лёгкая интеграция с REST API — React хорошо сочетается с REST-сервисами (GET, POST, PUT, DELETE), что удобно для работы с серверной частью приложения, обрабатывающей события календаря.

6. Горячая перезагрузка и удобная разработка — встроенные инструменты позволяют видеть изменения в реальном времени без полной перезагрузки страницы, что ускоряет разработку и тестирование.

7. Большое сообщество и поддержка — React имеет широкое сообщество, отличную документацию и множество готовых решений, что снижает порог входа и ускоряет решение проблем.

8. Интерактивный интерфейс — React позволяет создавать отзывчивый и удобный UI с анимациями, подтверждениями, всплывающими окнами и другими элементами, улучшающими пользовательский опыт при работе с событиями и календарём.

Применение фреймворка для Java **Spring Boot** имеет свои положительные аспекты:

1. Быстрый старт и автоматическая конфигурация — Spring Boot упрощает начальную настройку проекта благодаря автоконфигурации и встроенным зависимостям, что особенно удобно при локальной разработке.

2. Поддержка архитектуры MVC — встроенный Spring MVC позволяет чётко разделить контроллеры, бизнес-логику и доступ к данным, что делает код понятным и структурированным.

3. Гибкая работа с REST API — легко создавать REST-контроллеры для обработки запросов (GET, POST, PUT, DELETE), что идеально подходит для обмена событиями между клиентом и сервером.

4. Встроенная поддержка CORS — можно легко настроить CORS-фильтр для разрешения запросов с клиентской части, особенно если фронтенд запущен на другом порту (localhost:3000 ↔ localhost:8080).

5. Интеграция с базами данных — Spring Data JPA обеспечивает удобную работу с PostgreSQL, упрощая реализацию CRUD-операций для сущностей (пользователи, события, участники).

6. Тестируемость — Spring Boot отлично поддерживает модульное и интеграционное тестирование с JUnit и Mockito, что повышает надёжность серверной части.

7. Безопасность — при необходимости можно легко подключить Spring Security для аутентификации и авторизации пользователей.

8. Масштабируемость и модульность — приложение, изначально запущенное на локальном сервере, можно легко развернуть на сервер или в облаке без значительных изменений в архитектуре.

9. Активное сообщество и поддержка — большое количество документации и готовых решений упрощает разработку и устранение возможных проблем.

Spring Boot позволяет быстро и удобно создавать серверную часть приложения благодаря автоматической конфигурации, поддержке MVC и простому созданию REST API. Он упрощает взаимодействие с базой данных, легко настраивает CORS для связи с фронтендом и обеспечивает структурированный, тестируемый и масштабируемый код, что особенно удобно при разработке на локальном сервере.

Для разработки клиент-серверных приложений для планирования и организации мероприятий выбрана реляционная система управления базами данных **PostgreSQL**, которая имеет несколько ключевых преимуществ, особенно по сравнению с другими базами данных:

1. Надёжность и стабильность — PostgreSQL широко используется в продакшене и обеспечивает высокую устойчивость к сбоям;
2. Поддержка сложных связей — благодаря реляционной модели и поддержке внешних ключей легко реализовать связи между событиями, пользователями и регистрациями;
3. Мощные возможности фильтрации и запросов — язык SQL в PostgreSQL позволяет реализовать гибкие выборки по дате, типу событий, участникам и т. д.;
4. Расширяемость — при необходимости можно подключать дополнительные расширения для работы с JSON, гео Данными, полнотекстовым поиском;
5. Совместимость с фреймворками — PostgreSQL легко интегрируется с Spring Boot через JPA/Hibernate, обеспечивая удобную работу с базой в рамках MVC-архитектуры.
6. Консистентность и надёжность - PostgreSQL гарантирует соблюдение принципа ACID (Atomicity, Consistency, Isolation, Durability) для транзакций, что означает, что все операции над базой данных выполняются безопасно и надёжно. Это критично для приложений, где важна целостность данных, например, для управления регистрацией участников.
7. Открытый исходный код - PostgreSQL — это система с открытым исходным кодом, что означает, что она доступна бесплатно и имеет широкое сообщество разработчиков. Это позволяет снизить затраты на лицензирование и получение технической поддержки, а также использовать любые необходимые расширения и модификации под конкретные нужды проекта.

8. Интеграция с другими технологиями - PostgreSQL хорошо интегрируется с различными языками программирования и фреймворками, с инструментами для контейнеризации, такими как Docker. Это делает его универсальным выбором для разработки современных клиент-серверных приложений.

Благодаря своей надежности, гибкости и поддержке сложных запросов, PostgreSQL является эффективным решением для клиент-серверных приложений по планированию мероприятий.

2.3. Проектирование общей архитектуры

Проект основан на архитектуре клиент-сервер с разделением на клиентскую и серверную части, взаимодействующих через REST API по протоколу HTTP с использованием формата данных JSON. Приложение построено по модели MVC: контроллеры обрабатывают запросы, сервисы содержат бизнес-логику, а модели представляют структуру данных. Клиентская часть работает на React и запускается по адресу `http://localhost:3000`, обеспечивая взаимодействие пользователя с интерфейсом. Серверная часть реализована на Spring Boot и доступна на `http://localhost:8080`, отвечая за обработку запросов, авторизацию, управление данными и работу с базой данных.

В проекте используется **Docker** для контейнеризации всех компонентов системы, что обеспечивает воспроизводимость и удобство развертывания. Клиентская часть на React и серверная часть на Spring Boot упакованы в отдельные контейнеры с помощью Dockerfile, где задаются образы, зависимости и команды сборки.

С помощью `docker-compose.yml` описывается вся инфраструктура — клиентская часть, серверная часть и база данных PostgreSQL — с указанием портов, переменных окружения и сетевых настроек. Команда `docker-compose up --build` запускает все сервисы в нужной последовательности, объединяя их в единую сеть.

В результате клиент доступен по адресу `http://localhost:3000`, сервер — `http://localhost:8080`, а компоненты взаимодействуют по внутренним DNS-именам. Контейнеризация упрощает настройку, обеспечивает изоляцию, масштабируемость и стабильную работу в разных средах[3].

Безопасность в приложении реализована с использованием современных механизмов защиты. Аутентификация осуществляется через JWT (JSON Web Token), который сохраняется в локальном хранилище браузера и автоматически прикрепляется к каждому запросу для подтверждения личности пользователя. Пароли шифруются с использованием алгоритма `bcrypt`, что обеспечивает безопасное хранение данных. Для разграничения доступа применяется Spring Security, который управляет доступом к защищённым ресурсам и реализует ролевую модель с различными уровнями прав[7]. Кроме того, для обеспечения безопасного обмена между клиентом и сервером, расположенными на разных портах (`localhost:3000` и `localhost:8080`), используется CORS-фильтр, который позволяет принимать запросы только с доверенного домена (`http://localhost:3000`), блокируя запросы с других источников.

Для отслеживания работы системы используется библиотека `Log4j`, которая ведёт журнал событий, включая действия администраторов и возникающие ошибки. Логирование организовано по уровням важности, например, `INFO`, `DEBUG`, `ERROR`, что позволяет эффективно анализировать как повседневную активность, так и критические сбои.

Тестирование компонентов приложения выполняется с помощью `JUnit` и `Mockito`. Эти инструменты позволяют создавать модульные тесты для REST-контроллеров и сервисных классов, проверяя корректность бизнес-логики, безопасность и поведение системы в различных сценариях.

2.3.1. Разработка архитектуры клиентской части приложения

Интерфейс приложения разработан с использованием следующих технологий: **React, JavaScript, HTML, CSS, React Router, Axios, localStorage**.

React используется для построения интерфейса, включая отображение компонентов, календаря, форм событий, UI уведомлений. Для взаимодействия с сервером применяется библиотека **Axios**, обеспечивающая отправку HTTP-запросов к REST API. JWT-токен, используемый для аутентификации и авторизации, сохраняется в браузерном хранилище **localStorage**. Разметка страниц реализована с использованием **HTML**, а визуальное оформление — с помощью **CSS**.

Принцип работы системы заключается в следующем:

Пользователь заходит на клиентское приложение <http://localhost:3000>.

Выполняет действия: авторизация, регистрация, создание событий и т.д.

React-форма отправляет данные через **Axios** на backend (POST, GET, PUT).

JWT из **localStorage** добавляется в Authorization-заголовок.

Ответы сервера отображаются в UI: новые события, ошибки, уведомления.

Таким образом, клиентская часть приложения, реализованная с использованием **React** и сопутствующих технологий, обеспечивает динамичное и интерактивное взаимодействие с пользователем, эффективную работу с REST API и безопасное хранение аутентификационных данных. Такой подход позволяет создать отзывчивый, удобный и функциональный интерфейс, полностью интегрированный с серверной логикой.

2.3.2. Разработка архитектуры серверной части

Серверная часть приложения реализована на языке **Java** с использованием фреймворка **Spring Boot**, обеспечивающего быстрое

развёртывание и гибкую конфигурацию. Архитектура построена по принципу многоуровневого разделения ответственности (Controller - Service - Repository).

Для взаимодействия с реляционной базой данных **PostgreSQL** используется **Spring Data JPA**, реализующий ORM-модель и упрощающий доступ к данным. Безопасность системы обеспечивается с помощью **Spring Security**: реализована проверка JWT-токенов, контроль прав доступа к endpoint'ам, а также фильтрация CORS-запросов. Все пользовательские пароли хранятся в зашифрованном виде с использованием алгоритма bcrypt. Для идентификации и управления сессиями используются JWT-токены, внедряемые в HTTP-заголовки.

Централизованное логирование и упрощённая отладка реализованы через **Log4j**, а для обеспечения масштабируемой событийной архитектуры внедрён брокер сообщений **Apache Kafka**. Kafka используется для передачи внутренних событий системы — таких как приглашения пользователей, напоминания или уведомления об изменениях в событиях. Это позволяет отделить бизнес-логику от логики уведомлений, улучшить отклик сервера и упростить масштабирование.

Общая схема обработки запроса на сервере:

1. Клиент отправляет HTTP-запрос (например, POST /api/events).
2. Запрос проходит через **CORS-фильтр**, проверяющий допустимые источники (Origin), например http://localhost:3000.
3. **Spring Security**:
 - извлекает и валидирует JWT-токен;
 - проверяет права доступа и срок действия токена;
 - ограничивает доступ к защищённым ресурсам.
4. Контроллер обрабатывает запрос, делегируя логику в сервисный слой.
5. Сервисный слой выполняет:
 - валидацию DTO и бизнес-логику;

- вызовы к слоям данных (репозиториям).

6. Репозиторий взаимодействует с **PostgreSQL** через Spring Data JPA, обеспечивая транзакционность и соблюдение принципов ACID.

7. Результат возвращается клиенту в формате JSON, сопровождаясь соответствующим HTTP-статусом.

Контроллеры

AuthController

Назначение: обработка аутентификации и регистрации пользователей.

Зависимости: UserService, UserAuthenticationProvider. Эндпоинты описаны в табл.1.

Эндпоинты AuthController

Табл.1.

Метод	Путь	Действие	Входные данные	Возвращаемое значение
POST	/	Создание события	EventDto (@Valid) + JWT	201 Created с созданным EventDto
PUT	/ {eventId}	Обновление события по ID	EventDto + eventId + JWT	200 OK с обновленным EventDto
DELETE	/ {eventId}	Удаление события по ID	eventId + JWT	204 No Content
GET	/	Список событий пользователя	JWT	200 OK с List<EventDto> или 401
GET	/ {eventId}	Получение события по ID	eventId + JWT	200 с EventDto или 401

EventController

Назначение: управление событиями (CRUD-операции). Путь: /main/events,

зависимость: EventService. Эндпоинты приведены в табл.2.

Особенности:

- Проверка аутентификации через isUserDtoIsNull();
- Ошибки аутентификации → 401 Unauthorized;

- DTO валидируются через @Valid.

Эндпоинты EventController

Табл.2.

Метод	Путь	Действие	Входные данные	Возвращаемое значение
POST	/main/events	Создание события	EventDto (валидация), JWT	201 Created + данные события
PUT	/main/events/{eventId}	Обновление события по ID	EventDto (валидация), JWT, eventId	200 OK + обновлённое событие
DELETE	/main/events/{eventId}	Удаление события по ID	JWT, eventId	204 No Content
GET	/main/events	Получение событий пользователя	JWT	200 OK + список событий
GET	/main/events/{eventId}	Получение события по ID	JWT, eventId	200 OK + данные события

ParticipantsController

Назначение: управление участниками событий. Путь: /main/participants, зависимость: ParticipantService. Эндпоинты указаны в табл.3.

Эндпоинты ParticipantsController

Табл.3.

Метод	Путь	Действие	Входные данные	Возвращаемое значение
POST	/	Добавление участника	ParticipantDto (валидация @Valid) + JWT	201 Created с данными участника
DELETE	/eventId/login	Удаление участника	ID события и логин пользователя	204 No Content

UserController

Назначение: управление профилем пользователя. Путь: /user. Эндпоинты указаны в табл.4.

Метод	Путь	Действие	Входные данные	Возвращаемое значение
GET	/	Получение информации о пользователе	JWT-токен	200 OK с UserDto или 401
PUT	/	Обновление данных пользователя	UserDto с новыми данными + JWT	Обновленный UserDto

Сервисный слой

EventService

Назначение: реализация бизнес-логики по управлению событиями.

Зависимости: EventRepository, EventMapper, UserService, ParticipantService.

Методы указаны в табл.5.

Особенности:

- Использование @Transactional для модифицирующих операций;
- Автоматическое добавление организатора.

Методы EventService

Табл.5.

Метод	Параметры	Возвращаемое значение	Логика
findById	Long id	Event	Поиск события по ID
createEvent	Long userId, EventDto	EventDto	Создает событие и автоматически добавляет организатора
updateEvent	EventDto, long eventId	EventDto	Обновляет данные события (транзакционно)
deleteEvent	Long eventId	void	Удаляет событие
getEventById	Long userId	EventDto	Получает событие по ID пользователя
getEventByIdAndUserId	Long userId, Long eventId	EventDto	Получает событие с проверкой прав пользователя
getEventsByUserIdWithRoles	Long userId	List<EventDto>	Список событий пользователя с ролями
getEventsByUserId	Long userId	List<EventDto>	Список событий пользователя

UserService

Назначение: управление пользователями и аутентификацией. Зависимости: UserRepository, PasswordEncoder, UserMapper. Методы приведены в табл.6.

Особенности:

- Проверка уникальности логина;
- Генерация исключений ApplicationException.

Методы UserService

Табл.6.

Метод	Параметры	Возвращаемое значение	Логика
login	CredentialsDto	UserDto	Проверяет логин/пароль, возвращает DTO
register	SignUpDto	UserDto	Регистрирует нового пользователя (хеширует пароль)
findByLogin	String login	User	Поиск пользователя по логину
getUserById	Long id	User	Поиск пользователя по ID

ParticipantService

Назначение: работа с участниками событий. Зависимости: EventService, UserService, ParticipantMapper, ParticipantRepository, UserInvitedProducer.

Методы приведены в табл.7.

Особенности:

- Поддержка транзакций (@Transactional);
- Присвоение роли PARTICIPANT автоматически.

Методы ParticipantService

Табл.7.

Метод	Параметры	Возвращаемое значение	Логика
addParticipantToEvent	ParticipantDto	ParticipantDto	Добавляет участника к событию, отправляет уведомление
deleteParticipantFromEvent	Long eventId, String login	Void	Удаляет участника из события
findParticipantByUserIdAndEventId	Long userId, Long eventId	Participant	Находит участника по связке user/event

Репозиторный слой

Назначение: взаимодействие с базой данных через **Spring Data JPA**, реализация CRUD-операций, выполнение запросов и генерация исключений.

EventRepository

- Работа с сущностью Event;
- Методы: save, findById, findByUserId, findEventsWithRolesByUserId.

ParticipantRepository

- Работа с участниками событий (Participant);
- Методы: findByIdAndUserLogin, findByIdAndUserId.

UserRepository

- Работа с пользователями (User);
- Особенности: getByLogin выбрасывает ApplicationException, findByLogin возвращает Optional.

Все репозитории:

- Наследуют JpaRepository;
- Используют стандартные методы (save, delete, findById);
- Применяют DSL-запросы по именам методов.

Исключения:

- RuntimeException для событий/участников;
- ApplicationException с HTTP-статусами для пользователей.

2.3.3. Проектирование структуры базы данных

Для обеспечения надежного хранения, обработки и структурирования информации в приложении используется реляционная база данных PostgreSQL. Структура базы представлена таблицами: users, events, event_users (связывает участников с мероприятиями), notifications. Для обеспечения надежности операций соблюдаются ACID-свойства. Спроектированная база данных представлена на рис.2.

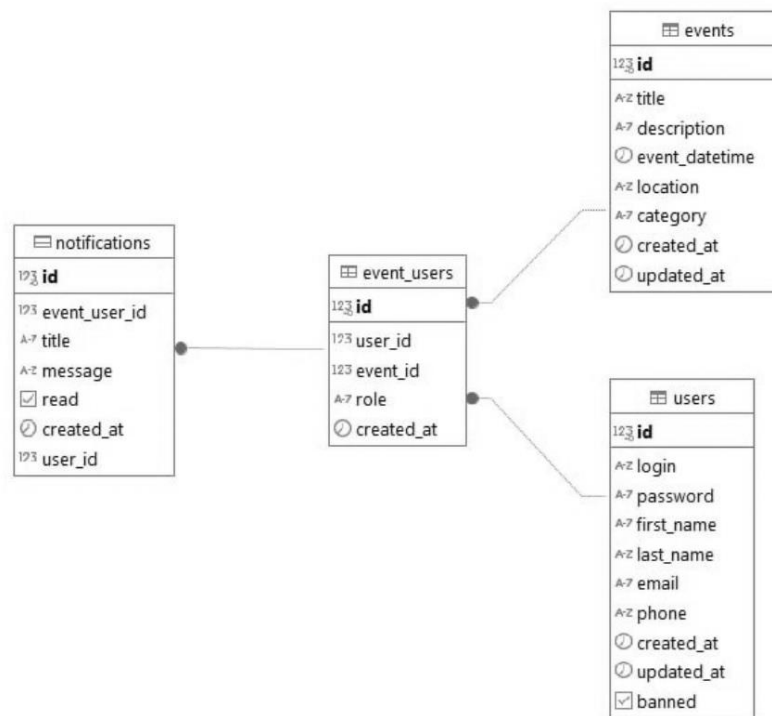


Рис.2. Структура БД

Основные таблицы и их структура.

1. **Пользователи (User)** — таблица пользователей: Содержит информацию о пользователях системы (логин, пароль, email, роль). Таблица включает уникальный идентификатор пользователя и служит точкой связей с другими сущностями.

Поля:

- id — уникальный идентификатор пользователя;
- first_name, last_name — имя и фамилия;
- login, password — данные для входа;
- phone — номер телефона;
- created_at, updated_at — временные метки создания и изменения записи;
- banned — статус блокировки пользователя (булевое значение).

2. **События (Event)** — таблица событий:

Хранят информацию о мероприятиях, создаваемых пользователями. Каждое событие связано с пользователем через внешний ключ user_id.

Поля:

- id — уникальный идентификатор
- title — название события
- description — описание
- event_datetime — дата и время
- location — место проведения
- category — категория
- created_at, updated_at — дата создания и обновления

3. **Участники(Event_users)** — таблица связи

пользователей(участников) с мероприятием:

Поля:

- id — уникальный идентификатор
- user_id — идентификатор пользователя
- event_id — идентификатор события
- role — роль участника в мероприятии (участник или организатор)
- created_at — дата создания

4. **Уведомления (Notification)** — таблица уведомлений:

Представляют напоминания о событиях. Связаны с событиями через внешний ключ event_id.

Поля:

- id — уникальный идентификатор
- event_user_id — идентификатор участника в мероприятии
- title — описание уведомления
- message — текст уведомления
- read — статус (проверка на прочтение уведомления)
- created_at — дата создания

В приложении реализованы следующие связи между таблицами:

- Один пользователь может создавать несколько событий (связь User → Event, один-ко-многим);

- Пользователь может принимать участие в нескольких событиях, а каждое событие — включать нескольких участников (связь User ↔ Event через Event_users, многие-ко-многим);

- Каждое участие пользователя в событии (Event_users) может быть связано с несколькими уведомлениями (Event_users → Notification, один-ко-многим).

Эти связи обеспечивают логичную структуру хранения данных и позволяют эффективно обрабатывать связанные сущности в рамках бизнес-логики приложения.

Взаимодействие приложения с базой данных осуществляется в формате SQL-запросов, направляемых из серверной части приложения. Эти запросы обрабатываются средствами **Spring Data JPA**, которая абстрагирует ручную работу с SQL и автоматически формирует команды для чтения и изменения данных.

Входящие данные представляют собой SQL-запросы, сгенерированные на основе действий пользователя и логики приложения. К основным типам относятся:

- SELECT — выборка данных (например, получение списка событий или пользователей);

- INSERT — добавление новой записи в таблицу (создание события, регистрация пользователя);

- UPDATE — обновление существующих данных (редактирование события или уведомления);

- DELETE — удаление записи (удаление пользователя, события и т.п.).

Исходящие данные формируются в результате выполнения SQL-запросов и включают два основных типа информации: таблицы и строки данных, полученные в результате выборки, например, список событий, а также метаданные, отражающие успешность выполнения операций, такие как статус выполнения команды INSERT или количество затронутых строк.

На сервере эти данные преобразуются в объекты Java (DTO), а затем преобразуются в формат **JSON** с помощью встроенных механизмов Spring Boot. Это позволяет клиентской части (React) получать структурированные ответы.

В результате внедрения реляционной базы данных PostgreSQL и организации таблиц с чётко определённой структурой и связями, приложение обеспечивает надёжное хранение, согласованность и целостность информации. Использование механизма Spring Data JPA упрощает доступ к данным, абстрагируя ручную работу с SQL и позволяя сосредоточиться на бизнес-логике. Благодаря правильно выстроенным отношениям между сущностями и применению ACID-свойств, достигается высокая стабильность операций, а структура базы данных полностью соответствует функциональным требованиям приложения.

Вывод: на этапе проектирования были рассмотрены и обоснованно выбраны языки программирования, система управления базами данных, фреймворки, средства контейнеризации и инструменты тестирования, обеспечивающие надёжность, масштабируемость и удобство в сопровождении разрабатываемой системы.

В рамках архитектурного проектирования были сформулированы и учтены все необходимые функциональные и нефункциональные требования, направленные на обеспечение стабильной, эффективной и безопасной работы приложения. С учетом этих требований спроектированы клиентская и серверная части системы с чётким разделением логики, а также разработана структура базы данных с учётом взаимосвязей между сущностями и соблюдением принципов целостности, согласованности и отказоустойчивости.

Применённый технологический стек в совокупности с проработанной архитектурой позволил создать современное веб-приложение, обладающее высокой степенью надёжности, устойчивости и защищённости, а также адаптированное для дальнейшего масштабирования и сопровождения.

3. РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ

В данном разделе подробно рассмотрена реализация ключевых функциональных требований приложения, направленных на обеспечение удобного взаимодействия пользователя с системой. Особое внимание уделено описанию механизмов регистрации и авторизации пользователей, создания и редактирования мероприятий, добавления участников и организаторов, отправки уведомлений и напоминаний, а также инструменты администрирования системы. Структура проекта приведена на рис.3.

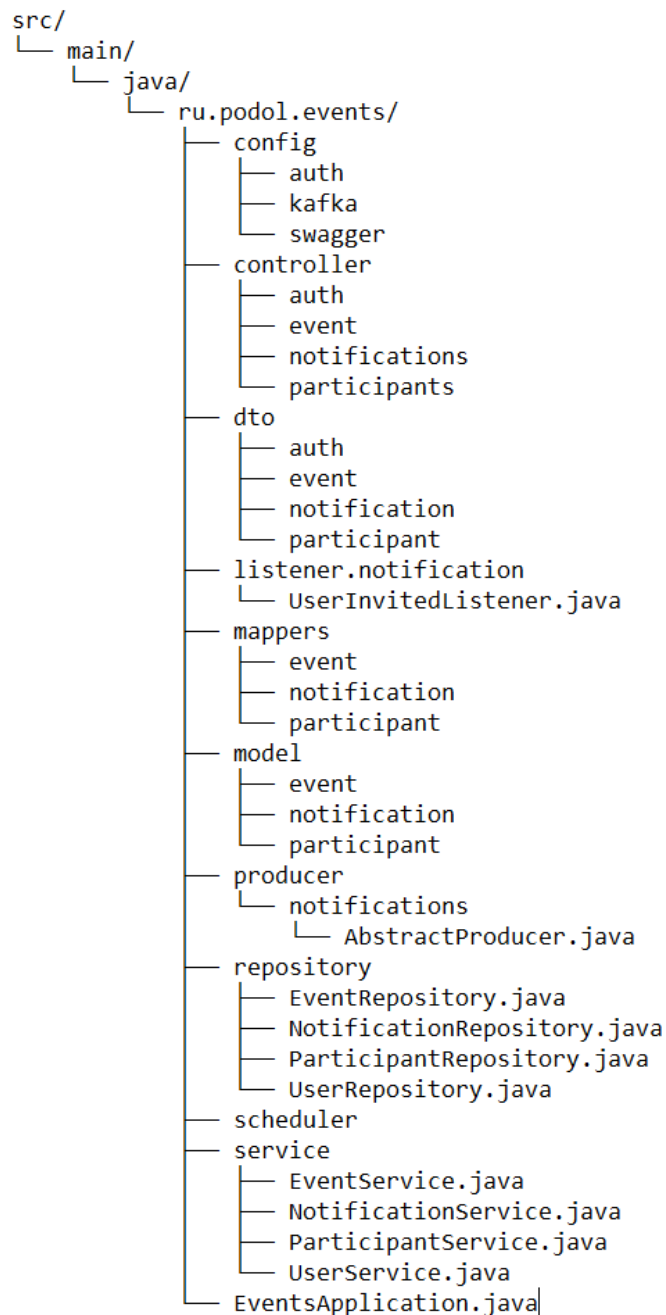


Рис.3. Структура проекта

3.1. Регистрация пользователей

Позволяет пользователям создать учетную запись и получить доступ к функционалу приложения. Поле для регистрации приведено на рис.4.

Регистрация пользователей включает следующие этапы:

1) заполнение формы на клиентской стороне: пользователь вводит имя, фамилию, email и пароль. Регистрационная форма реализована с использованием фреймворка React. Данные проходят первичную валидацию и отправляются на сервер через REST API:

```
@PostMapping("/register")
public ResponseEntity<UserDto> register(@RequestBody @Valid SignUpDto
user) {
    UserDto createdUser = userService.register(user);

    createdUser.setToken(userAuthenticationProvider.createToken(user.getLogin()));
    return ResponseEntity.created(URI.create("/users/" +
createdUser.getId())).body(createdUser);
}
```

2) валидация данных: проверяется, что email уникален (не зарегистрирован ранее) путём запроса к базе данных PostgreSQL. Пароль проверяется на соответствие требованиям безопасности (например, длина, наличие символов и цифр) на стороне сервера, реализованного с использованием Spring Boot;

3) шифрование пароля: перед сохранением в базу данных пароль шифруется с помощью алгоритма BCryptPasswordEncoder, чтобы исключить его хранение в открытом виде;

4) создание учетной записи

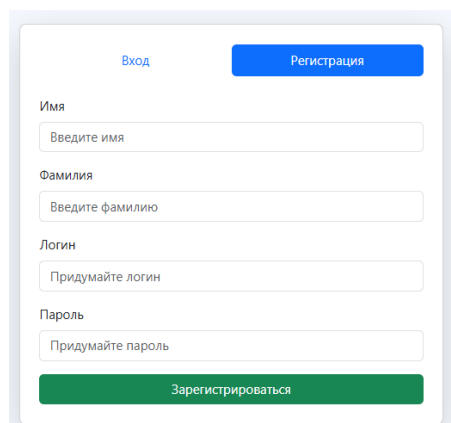
The image shows a web form for user registration. At the top, there are two buttons: 'Вход' (Login) in blue and 'Регистрация' (Registration) in blue. Below these are four input fields: 'Имя' (Name) with placeholder 'Введите имя', 'Фамилия' (Surname) with placeholder 'Введите фамилию', 'Логин' (Login) with placeholder 'Придумайте логин', and 'Пароль' (Password) with placeholder 'Придумайте пароль'. At the bottom of the form is a green button labeled 'Зарегистрироваться' (Register).

Рис.4. Поле для регистрации

Схема алгоритма работы приведена в Приложении Б (рис.Б.2.).

3.2. Авторизация пользователей

Позволяет зарегистрированным пользователям получить доступ к системе с использованием их учетных данных (email и пароль). Поле для авторизации представлено на рис.5.

Рис.5. Поле для авторизации

Авторизация пользователей включает следующие этапы:

1) обработка запроса на авторизацию (Backend): пользователь отправляет email и пароль на маршрут /login. Серверная часть, реализованная на Spring Boot, проверяет наличие email в базе данных PostgreSQL и сравнивает введенный пароль с хешированным значением с помощью BCryptPasswordEncoder.

```
@PostMapping("/login")
public ResponseEntity<UserDto> login(@RequestBody @Valid CredentialsDto
credentialsDto) {
    UserDto userDto = userService.login(credentialsDto);

    userDto.setToken(userAuthenticationProvider.createToken(userDto.getLogin()));
    return ResponseEntity.ok(userDto);
}
```

В случае успеха генерируется JWT-токен с использованием библиотеки java-jwt, включающий идентификатор пользователя, его роль и срок действия. Токен возвращается клиенту;

2) работа с токеном на клиенте (React + Axios): после успешной авторизации JWT сохраняется в localStorage. Все последующие запросы отправляются с токеном в заголовке Authorization, который добавляется с помощью Axios. Это позволяет обращаться к защищенным маршрутам без повторной авторизации;

3) проверка токена на сервере: при каждом запросе к защищённому ресурсу сервер извлекает JWT из заголовка, проверяет его подлинность и срок действия. Если токен валиден, пользователь получает доступ. В противном случае возвращается статус 401 Unauthorized.

Схема алгоритма работы приведена в Приложении Б (рис.Б.3.).

3.3. Управление мероприятиями

Создание мероприятия.

Создание мероприятия реализовано с использованием Spring Boot, JPA, PostgreSQL на серверной стороне и React на клиентской стороне.

Процесс включает следующие этапы:

- 1) авторизованный пользователь (с JWT) отправляет POST-запрос на /main/events с данными: название, описание, дата, тип мероприятия, место, и список участников (email);
- 2) сервер (Spring Boot) создает объект Event, устанавливает текущего пользователя как организатора, сохраняет данные в таблицу events через JPA;
- 3) участники и организаторы добавляются в таблице event_users;
- 4) все данные сохраняются в PostgreSQL, доступ контролируется через Spring Security.

```
@PostMapping
public ResponseEntity<?> createEvent(@AuthenticationPrincipal UserDto
userDto,
                                   @RequestBody @Valid EventDto eventDto) {
    if (isUserDtoIsNull(userDto)) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("User not
authenticated");
    }

    EventDto createdEvent = eventService.createEvent(userDto.getId(),
eventDto);
    return ResponseEntity.status(HttpStatus.CREATED).body(createdEvent);
}
```

Меню для создания мероприятий представлены на рис.6, 7.

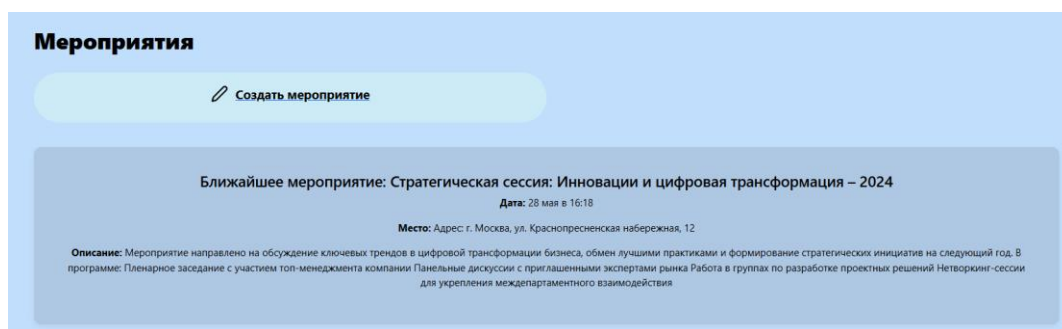


Рис.6. Создание мероприятия (стартовое меню).

Создать мероприятие

Название мероприятия:
Введите название

Описание:
Введите описание мероприятия

Дата и время:
ДД.ММ.ГГГГ --:--

Место проведения:
Введите место

Категория:
Конференция

Создать

Рис.7. Создание мероприятия (форма для ввода).

Схема алгоритма работы приведена в Приложении Б (рис.Б.4.).

Редактирование мероприятия.

Редактирование мероприятия реализовано с использованием Spring Boot, JPA, PostgreSQL и React.

Процесс включает следующие этапы:

- 1) пользователь отправляет PUT-запрос с обновлёнными данными через клиент на React;
- 2) сервер (Spring Boot) проверяет права доступа (организатор);

3) обновляются поля события (дата, место, описание), участники и организаторы;

4) изменения сохраняются в базе данных через JPA;

5) безопасность обеспечивается с помощью Spring Security и JWT.

```
@PutMapping("/{eventId}")
public ResponseEntity<?> updateEventById(@AuthenticationPrincipal UserDto
userDto,
                                           @RequestBody @Valid
EventDto eventDto,
                                           @PathVariable long
eventId) {
    if (isUserDtoIsNull(userDto)) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("User not
authenticated");
    }
    EventDto event = eventService.updateEvent(eventDto, eventId);
    return ResponseEntity.ok(event);
}
```

Схема алгоритма работы приведена в Приложении Б (рис.Б.6.).

Удаление мероприятия.

Удаление мероприятия реализовано с использованием Spring Boot, Spring Security, JPA и PostgreSQL.

Процесс включает следующие этапы:

- 1) клиент (React) отправляет DELETE-запрос на /main/events/{id};
- 2) сервер проверяет права доступа пользователя (организатор);
- 3) при подтверждении доступа событие и связанные с ним данные (участники, уведомления) удаляются из базы каскадно с помощью аннотаций JPA (@OneToMany (cascade = CascadeType.ALL)).

```
@DeleteMapping("/{eventId}")
public ResponseEntity<?> deleteEventById(@AuthenticationPrincipal UserDto
userDto,
                                           @PathVariable long
eventId) {
    if (isUserDtoIsNull(userDto)) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("User not
authenticated");
    }
    eventService.deleteEvent(eventId);
    return ResponseEntity.noContent().build();
}
```

Меню для редактирования или удаления мероприятия представлено на рис.8.

Стратегическая сессия: Инновации и цифровая трансформация – 2024

Название мероприятия:

Стратегическая сессия: Инновации и цифровая траи

Описание:

Панельные дискуссии с приглашенными экспертами рынка

Дата и время:

28.05.2025 16:18

Место проведения:

Адрес: г. Москва, ул. Краснопресненская набережнi

Категория:

Конференция

Участники

Логин участника

sergeymu2015@yandex.ru

testuser

Сохранить изменения

Удалить мероприятие

Рис.8. Редактирование или удаление мероприятия.

3.4. Календарь мероприятий

Календарь мероприятий реализован в виде последовательного списка предстоящих событий с возможностью их визуального отображения по дате и времени.

Технологии, использованные при реализации Spring Boot, JPA и PostgreSQL на серверной стороне и React на клиентской.

Процесс работы включает следующие этапы:

- 1) клиентская часть отправляет GET-запрос на /main/events с параметрами фильтрации (дата, статус, роль пользователя и т.д.);
- 2) серверная часть (Spring Boot) обрабатывает запрос, применяет фильтрацию на уровне JPA, извлекая соответствующие события из базы данных.

```

    @GetMapping
    public ResponseEntity<List<EventDto>>
    getEventsByUser(@AuthenticationPrincipal UserDto userDto) {
        if (isUserDtoIsNull(userDto)) {
            return
            ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Collections.emptyList());
        }
        List<EventDto> events =
        eventService.getEventsByUserIdWithRoles(userDto.getId());

        return ResponseEntity.ok(events);
    }

    @GetMapping("/{eventId}")
    public ResponseEntity<?> getEventById(@AuthenticationPrincipal UserDto
    userDto,
                                           @PathVariable long
    eventId) {
        if (isUserDtoIsNull(userDto)) {
            return
            ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Collections.emptyList());
        }
        EventDto event = eventService.getEventById(eventId);

        return ResponseEntity.ok(event);
    }

```

События упорядочиваются по дате и формируются в виде списка, который возвращается клиенту. Меню календаря мероприятий представлено на рис.9.

3) на клиенте (React) отображается плейлист с возможностью фильтрации и поиска, реализован интерфейс для навигации по событиям;

4) доступ к данным регулируется через Spring Security с учётом прав пользователя, роль которого определяет доступ к конкретным событиям.

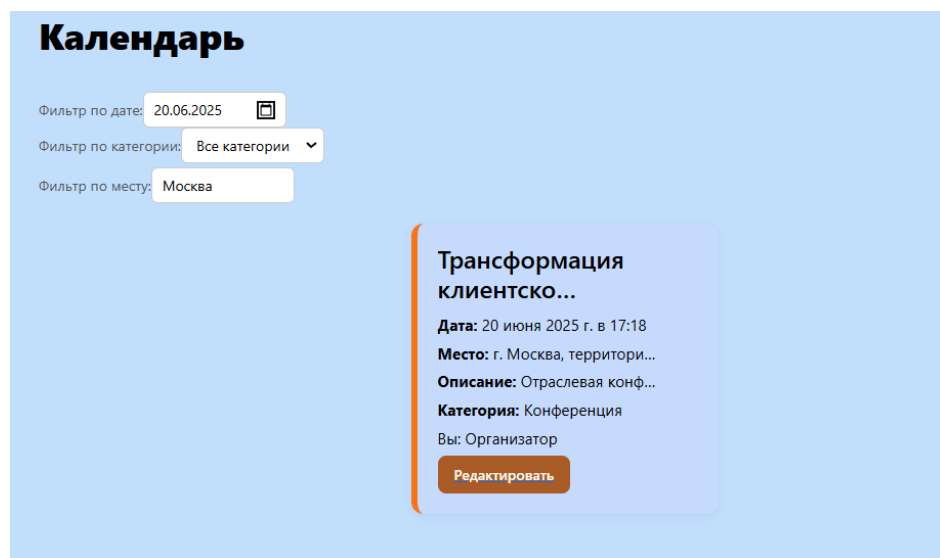


Рис.9. Отслеживание мероприятий у организаторов и участников.

Схема алгоритма работы приведена в Приложении Б (рис.Б.5.).

Система уведомлений и напоминаний

Уведомления о событиях и напоминания для участников реализованы с использованием Apache Kafka, планировщика задач и механизмов отображения уведомлений на клиентской стороне. Технологии, используемые при реализации: Spring Boot (серверная часть), Spring Kafka (интеграция с Kafka), JPA (доступ к данным), React (интерфейс пользователя), а также аннотация `@Scheduled` для реализации планировщика задач.

Процесс реализации включает следующие этапы:

1) во время приглашения участника на мероприятие сохраняется новая запись в базе данных. Параллельно в соответствующий Kafka-топик отправляется событие об отправке уведомления соответствующему пользователю:

```
@Transactional
public ParticipantDto addParticipantToEvent(ParticipantDto participantDto) {
    User user = userService.findByLogin(participantDto.getLogin());
    Event event = eventService.findById(participantDto.getEventId());
    log.info("Adding participant to event {}", event);
    Participant participant =
    participantMapper.toParticipant(participantDto);

    participant.setRole(UserEventRole.PARTICIPANT);
    participant.setUser(user);
    participant.setEvent(event);

    log.info("sending notify: adding participant to event {}", event);
    EventNotification userInvited = new EventNotification(
        user.getId(),
        "Вас добавили на мероприятие",
        NotificationType.INVITE,
        event.getTitle()
    );
    kafkaEventNotificationsProducer.sendMessage(userInvited);

    log.info("Participant {} added to event",
    participantMapper.toParticipantDto(participant));
    return
    participantMapper.toParticipantDto(participantRepository.save(participant));
}
```

2) планировщик задач (Task Scheduler) отслеживает приближение времени начала мероприятий:

```
@Component
@RequiredArgsConstructor
public class EventReminderScheduler {
```

```

public final ParticipantService participantService;

@Scheduled(cron = "${spring.notification.reminder.cron}")
public void remindToEvent() {
    participantService.send24HourReminders();
}
}

```

и публикует напоминания в Kafka-топик:

```

public void send24HourReminders() {
    List<Participant> participants = findParticipantsForReminder24Hours();

    participants.forEach(participant -> {
        notificationService.sendReminder(
            participant.getUser().getId(),
            "Напоминание о мероприятии",
            NotificationType.REMIND,
            String.format("Через %d часов: %s", REMIND_24_HOURS,
participant.getEvent().getTitle())
        );
        participant.setReminderSent(true);
    });

    participantRepository.saveAll(participants);
}

```

3) слушатели Kafka (Kafka consumers), работающие на стороне сервера, получают события из топиков и инициируют создание уведомлений в базе данных;

4) при входе пользователя в систему клиентская часть отправляет запрос на /main/notifications.

```

@GetMapping
public ResponseEntity<List<NotificationDto>>
getNotificationsByUserId(@AuthenticationPrincipal UserDto userDto) {
    if (isUserDtoIsNull(userDto)) {
        return
ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Collections.emptyList());
    }
    List<NotificationDto> notifications =
notificationService.getNotificationsByUserId(userDto.getId());

    return ResponseEntity.ok(notifications);
}

```

5) сервер возвращает список актуальных уведомлений, связанных с предстоящими, обновлёнными или отменёнными мероприятиями, в которых пользователь участвует;

6) на клиенте (React) уведомления отображаются в специальном разделе, обеспечивая своевременное информирование пользователя о событиях.

Меню уведомлений представлено на рис.10.

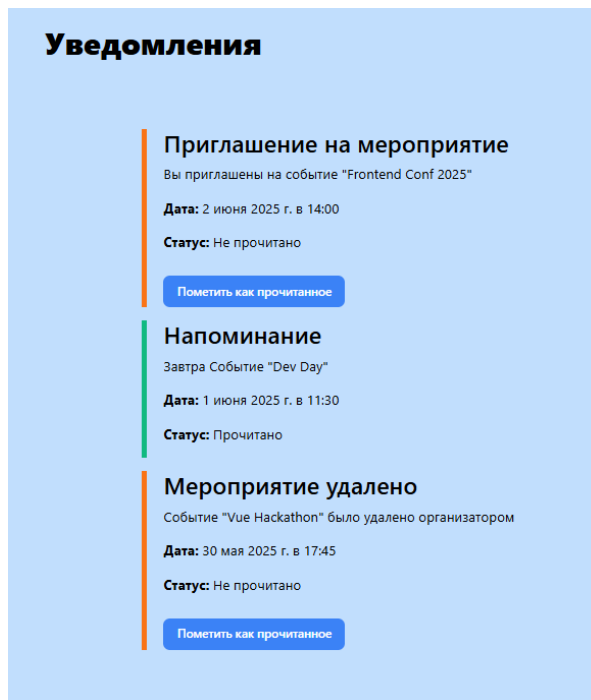


Рис.10. Меню уведомлений.

Схема алгоритма работы приведена в Приложении Б (рис.Б.7.).

3.5. Тестирование программы

В рамках тестирования было проведено модульное (unit) тестирование с использованием фреймворков JUnit и Mockito. Дополнительно осуществлялась валидация API-запросов с помощью Postman для проверки корректности взаимодействия между компонентами системы. Тесты приведены в Приложении Г. Результаты работы тестов приведены на рис.11, 12.

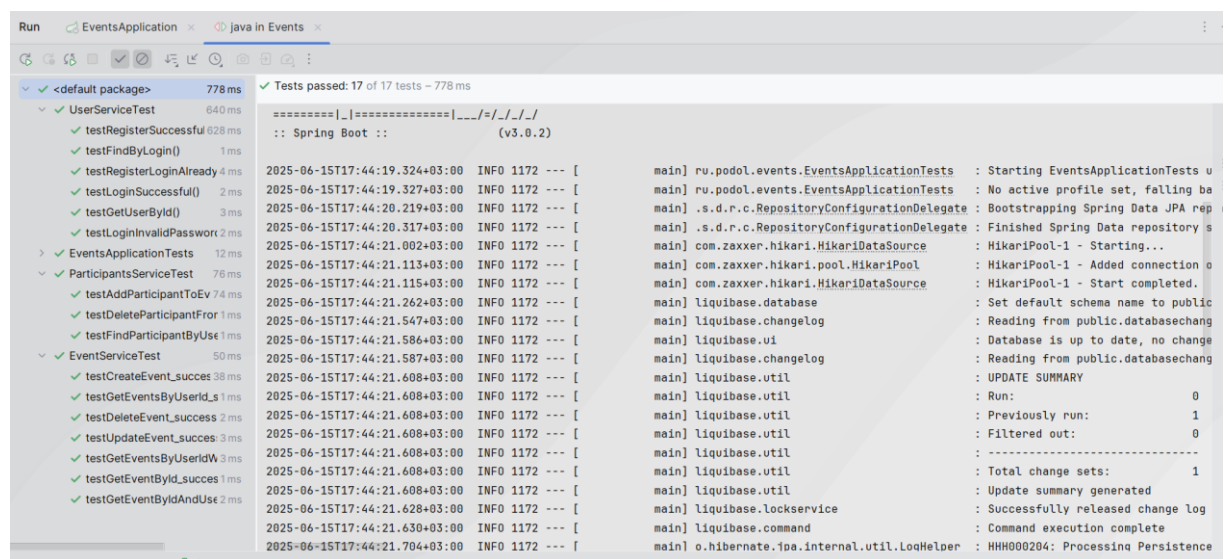


Рис.11. Тестирование функций приложения

✓ KafkaIntegr 1 sec 414 ms	✓ Tests passed: 1 of 1 test – 1 sec 414 ms
✓ testEver 1 sec 414 ms	2025-06-17T20:15:10.377+03:00 INFO 5336 --- [quest-handler-5] k.coordinator.grou 2025-06-17T20:15:10.378+03:00 INFO 5336 --- [cutor-Rebalance] k.coordinator.grou 2025-06-17T20:15:10.379+03:00 INFO 5336 --- [main] o.a.k.c.c.internal: 2025-06-17T20:15:10.379+03:00 INFO 5336 --- [main] o.a.k.c.c.internal: 2025-06-17T20:15:10.380+03:00 INFO 5336 --- [quest-handler-3] k.coordinator.grou 2025-06-17T20:15:10.381+03:00 INFO 5336 --- [ntainer#0-0-C-1] o.s.k.l.KafkaMessai 2025-06-17T20:15:10.382+03:00 INFO 5336 --- [main] o.a.k.c.c.internal: 2025-06-17T20:15:10.382+03:00 INFO 5336 --- [main] o.a.k.c.c.internal: 2025-06-17T20:15:10.382+03:00 INFO 5336 --- [main] o.a.k.c.c.internal: 2025-06-17T20:15:10.384+03:00 INFO 5336 --- [main] o.a.k.c.c.internal: 2025-06-17T20:15:10.391+03:00 INFO 5336 --- [main] o.a.k.c.c.internal: 2025-06-17T20:15:10.482+03:00 INFO 5336 --- [ntainer#0-0-C-1] r.p.e.l.n.UserInvi Уведомление обработалось

Рис.12. Тестирование отправки и обработки уведомлений

Вывод: в данной главе представлена реализация основных функциональных возможностей разработанного веб-приложения для планирования и организации мероприятий. Показано, как реализованы процессы регистрации и авторизации пользователей, управление мероприятиями, включая их создание, редактирование и удаление, а также добавление и удаление организаторов и участников. Описана реализация механизма редактирования места и времени проведения событий, интеграция календаря мероприятий, а также система уведомлений и напоминаний, обеспечивающая своевременное информирование пользователей. Кроме того, продемонстрированы элементы, обеспечивающие поддержку работы приложения, включая систему логирования для мониторинга и анализа его функционирования.

В итоге можно сделать вывод, что реализованное приложение соответствует заявленным функциональным требованиям, демонстрирует устойчивую и безопасную работу, и может служить основой для внедрения и последующего расширения системы в условиях реального использования.

ЗАКЛЮЧЕНИЕ

В рамках выпускной работы была спроектирована и реализована система клиент-серверного взаимодействия для управления событиями в формате календаря. Основной целью проекта является создание удобного приложения с возможностью работы в локальной среде.

Приложение построено с использованием современных технологий, таких как React для клиентской части и Spring Boot для серверной, что обеспечивает высокую производительность, удобство в использовании и безопасность данных. В процессе разработки было уделено особое внимание архитектуре приложения, что позволило обеспечить надёжную и эффективную работу всех компонентов системы.

Проектирование архитектуры основывалось на чётком разделении слоёв приложения: клиентский интерфейс взаимодействует с сервером через REST API, что позволяет эффективно и безопасно обрабатывать запросы. Серверная часть приложения отвечает за управление данными и их обработку, используя Spring Boot и базу данных PostgreSQL. Важным аспектом реализации было использование JWT для аутентификации и авторизации, а также внедрение механизмов безопасности, таких как bcrypt для шифрования паролей и CORS-фильтра для защиты от несанкционированных запросов.

База данных была спроектирована с учётом всех функциональных требований, и её структура позволяет эффективно хранить и управлять информацией о пользователях, событиях, участниках, организаторах и уведомлениях. Использование Spring Data JPA абстрагирует работу с SQL-запросами, позволяя сосредоточиться на логике приложения.

Реализация была выполнена с акцентом на удобство использования интерфейса и безопасность. Взаимодействие с сервером через клиентскую часть происходит с использованием современных библиотек и инструментов, таких как Axios и JWT. Интерфейс приложения разработан с учётом

потребностей конечных пользователей и с использованием принципов UI/UX-дизайна.

Реализованные функциональные и нефункциональные требования подтверждают готовность приложения к использованию и возможному расширению.

В перспективе дальнейшего развития клиент-серверного приложения можно реализовать множество функциональных и технических улучшений.

В дальнейшем приложение может быть существенно доработано и расширено за счёт внедрения дополнительных функциональных возможностей. Одним из перспективных направлений развития является интеграция с внешними календарными сервисами, такими как Google Calendar, Outlook Calendar и Apple Calendar. Это позволит синхронизировать события между приложением и сторонними системами, автоматически импортировать и экспортировать мероприятия, а также отправлять приглашения через почтовые сервисы, связанные с календарями.

Ещё одним направлением развития является расширение системы уведомлений. В будущем может быть реализована многоуровневая система оповещений с возможностью выбора каналов доставки (электронная почта, push-уведомления, SMS), гибкой настройкой времени напоминаний (например, за 10 минут, 1 час или 1 день до начала мероприятия), а также централизованным управлением уведомлениями через личный кабинет пользователя. Дополнительно может быть выполнена интеграция с Firebase для реализации мобильных push-уведомлений.

Для повышения персонализации и удобства использования возможно добавление расширенных пользовательских настроек. Это включает в себя поддержку различных тем оформления, включая ночной режим, настройку приватности отображения событий, а также управление участием в мероприятиях с возможностью подтверждения, отклонения или предложения изменений.

Особое внимание может быть уделено поддержке мобильных устройств. В перспективе возможно создание адаптивного интерфейса, а также разработка полноценного мобильного приложения на базе таких кроссплатформенных технологий, как React Native или Flutter. Это обеспечит удобный доступ к функционалу приложения с любых устройств и позволит использовать системные уведомления для своевременного информирования пользователей.

Наконец, полезным расширением станет внедрение системы аналитики и отчётности. Для организаторов мероприятий могут быть реализованы графики посещаемости, инструменты анализа вовлечённости участников, а также возможность экспорта отчётов в форматах PDF и Excel. Всё это позволит повысить эффективность планирования и контроля мероприятий, а также принимать более обоснованные управленческие решения.

Перечисленные направления показывают высокий потенциал для дальнейшего развития приложения как в функциональном, так и в техническом аспекте. Благодаря модульной архитектуре и использованному технологическому стеку реализация этих перспектив возможна поэтапно, без полной переработки системы. Это делает разработанное приложение не только актуальным, но и масштабируемым решением, способным эволюционировать в полноценную платформу для управления мероприятиями различного уровня сложности.

СПИСОК ЛИТЕРАТУРЫ

1. Spring Boot Documentation[Электронный ресурс].URL: <https://docs.spring.io/spring-boot/docs/current/reference/html/>(Дата обращения: 15.03.2025)
2. PostgreSQL Documentation[Электронный ресурс].URL: <https://www.postgresql.org/docs/>(Дата обращения: 15.03.2025)
3. Docker Documentation[Электронный ресурс].URL: <https://docs.docker.com/> (Дата обращения: 01.04.2025)
4. Java Platform, Standard Edition Documentation [Электронный ресурс]. – URL: <https://docs.oracle.com/javase/8/docs/> (дата обращения: 20.03.2025).
5. React Documentation [Электронный ресурс]. – URL: <https://react.dev/> (дата обращения: 25.03.2025).
6. Spring Data JPA Reference Documentation [Электронный ресурс]. – URL: <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/> (дата обращения: 27.03.2025).
7. Spring Security Reference [Электронный ресурс]. – URL: <https://docs.spring.io/spring-security/reference/index.html> (дата обращения: 30.03.2025).
8. Apache Kafka Documentation [Электронный ресурс]. – URL: <https://kafka.apache.org/documentation/> (дата обращения: 05.04.2025).
9. JUnit 5 User Guide [Электронный ресурс]. – URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 20.04.2025).
10. Mockito Documentation [Электронный ресурс]. – URL: <https://site.mockito.org/> (дата обращения: 03.06.2025).
11. Стефанов С. Реакт. Быстрый старт. 2-е изд. – СПб.: Питер, 2023. – 304 с.
12. Моргунов Е. П. PostgreSQL. Основы языка SQL: учеб. пособие / Е. П. Моргунов; под ред. Е. В. Рогова, П. В. Лузанова. – СПб.: БХВ-Петербург, 2018. – 336 с.

ПРИЛОЖЕНИЕ А

Задание на выпускную квалификационную работу.



МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное
учреждение высшего образования
«Национальный исследовательский университет «МЭИ»

Институт
Кафедра

ИВТИ
ВМСС

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ (бакалаврскую работу)

Направление 09.03.01 Информатика и вычислительная техника
(код и наименование)

Направленность (профиль)

Вычислительные машины, комплексы, системы и сети

Форма обучения Очная
(очная/очно-заочная/заочная)

Тема: Разработка клиент-серверного приложения для планирования и организации мероприятий

Студент А-08-21 Муромцев С.А.
группа подпись фамилия и инициалы

Научный руководитель К.Т.Н. доцент Раскатова М.В.
уч. степень должность подпись фамилия и инициалы

Консультант уч. степень должность подпись фамилия и инициалы

Консультант уч. степень должность подпись фамилия и инициалы

Зав. кафедрой К.Т.Н. доцент Вишняков С.В.
уч. степень звание подпись фамилия и инициалы

Место выполнения работы НИУ «МЭИ», каф. ВМСС

СОДЕРЖАНИЕ РАЗДЕЛОВ ЗАДАНИЯ И ИСХОДНЫЕ ДАННЫЕ

В современных условиях, когда эффективное планирование мероприятий и управление ими являются ключевыми аспектами успешной работы организаций и личных инициатив, возникает потребность в удобных цифровых решениях. Клиент-серверное приложение для планирования и организации мероприятий станет полезным инструментом как для организаторов, так и для участников. Оно позволит создавать и управлять событиями, регистрировать участников, отслеживать расписание и отправлять уведомления.

Приложение будет разработано с использованием следующих технологий:

- Spring Boot для реализации серверной части и предоставления RESTful API.
- PostgreSQL как основной базы данных для хранения информации о мероприятиях, пользователях и регистрациях.
- Docker для контейнеризации приложения, что обеспечит простое развертывание и масштабируемость.

Основные функции приложения:

1. Создание и управление мероприятиями: добавление информации о событиях, включая название, описание, дату, время и место проведения.
2. Регистрация участников: возможность организаторам приглашать участников на мероприятия.
3. Отслеживание расписания: просмотр всех мероприятий с фильтрацией по дате, категории и местоположению.
4. Поддержка многопользовательской работы: возможность для организаторов управлять своими мероприятиями и для участников следить за их статусом.

Для реализации приложения потребуется:

- Графический интерфейс: удобный и интуитивно понятный интерфейс для работы с мероприятиями.
- Серверная часть: реализация бизнес-логики и API для взаимодействия с клиентами.
- Интеграция с базой данных: надежное хранилище данных для участников и мероприятий.
- Контейнеризация через Docker: упрощение деплоя и обеспечение модульности приложения.

ПЕРЕЧЕНЬ ГРАФИЧЕСКОГО МАТЕРИАЛА

1. Схема алгоритма регистрации и входа пользователей: Процесс аутентификации и авторизации.

2. Схема добавления и редактирования мероприятий: Логика создания, обновления и удаления мероприятий.

3. Пример интерфейса для создания нового мероприятия: Визуализация формы ввода данных.

4. Пример интерфейса отображения расписания мероприятий: Варианты представления событий в виде списка.

Количество листов	4
Количество слайдов в презентации	Не менее 8

РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА

1. Spring Boot Documentation[Электронный ресурс].URL: <https://docs.spring.io/spring-boot/docs/current/reference/html/>(Дата обращения: 13.12.2024)
2. PostgreSQL Documentation[Электронный ресурс].URL: <https://www.postgresql.org/docs/>(Дата обращения: 13.12.2024)
3. Redis Documentation[Электронный ресурс].URL: <https://redis.io/docs/latest/> (Дата обращения: 13.12.2024)
4. Docker Documentation[Электронный ресурс].URL: <https://docs.docker.com/> (Дата обращения: 13.12.2024)

Примечания:

1. Задание брошюруется вместе с выпускной работой после титульного листа (страницы задания имеют номера 2, 3).
2. Отзыв руководителя, рецензия(и), отчет о проверке на объем заимствований и согласие студента на размещение работы в открытом доступе вкладываются в конверт (файловую папку) под обложкой работы.

ПРИЛОЖЕНИЕ Б

Схемы работы приложения

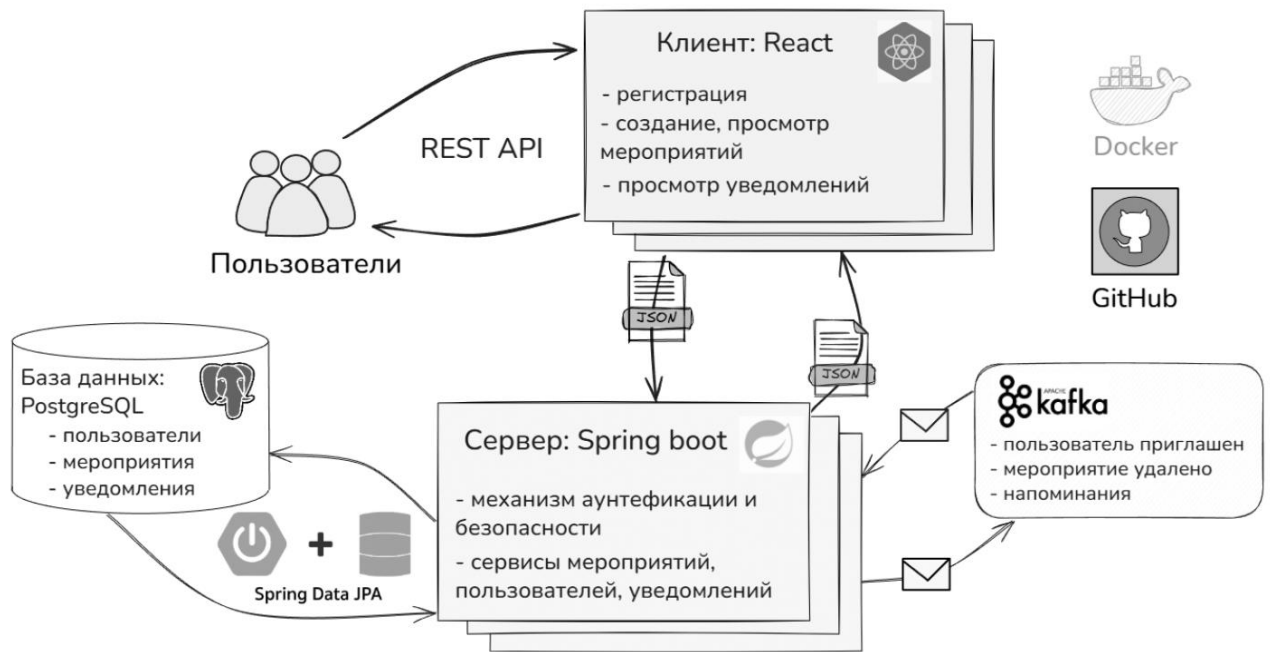


Рис.Б.1. Общая схема работы приложения.



Рис.Б.2. Схема алгоритма работы регистрации.



Рис.Б.3. Схема алгоритма работы авторизации.



Рис.Б.4. Схема алгоритма создания мероприятия.

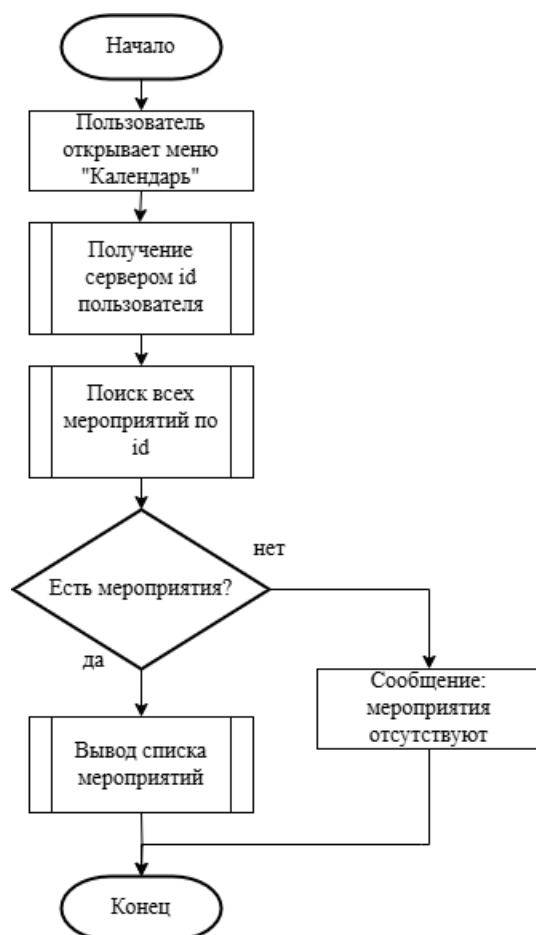


Рис.Б.5. Схема алгоритма просмотра мероприятий.



Рис.Б.6. Схема алгоритма редактирования мероприятия.

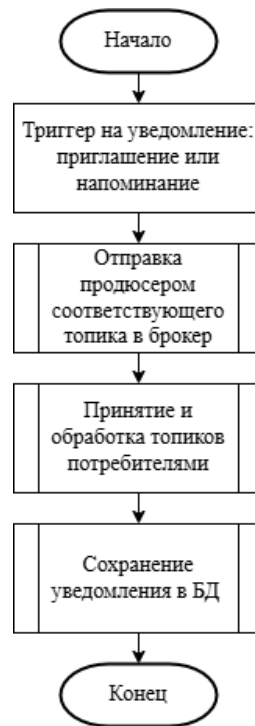


Рис.Б.7. Схема алгоритма создания уведомления.

ПРИЛОЖЕНИЕ В

Листинг кода

Листинг серверной логики (Spring boot).

Сервисы:

```
/*
Назначение: сервисный слой, обеспечивающий бизнес-логику для работы с
пользователями – регистрация, авторизация, получение информации
Входные данные: учетные данные, DTO пользователей, идентификаторы
Результат: возврат данных о пользователе или выполнение операций с ними
ФИО: Муромцев Сергей Андреевич
Дата написания: 30.05.2025 г.
Версия: 1.0
*/
@Service
@RequiredArgsConstructor
public class UserService {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final UserMapper userMapper;

    // Авторизация пользователя по логину и паролю
    public UserDto login(CredentialsDto credentialsDto) {
        User user =
userRepository.getByLogin(credentialsDto.getLogin());

        if
(passwordEncoder.matches(CharBuffer.wrap(credentialsDto.getPassword()),
user.getPassword())) {
            return userMapper.toUserDto(user);
        }
        throw new AppException("Invalid password",
HttpStatus.BAD_REQUEST);
    }

    // Регистрация нового пользователя
    public UserDto register(SignUpDto userDto) {
        Optional<User> optionalUser =
userRepository.findByLogin(userDto.getLogin());
        if (optionalUser.isPresent()) {
            throw new AppException("Login already exists",
HttpStatus.BAD_REQUEST);
        }

        User user = userMapper.signUpToUser(userDto);

        user.setPassword(passwordEncoder.encode(CharBuffer.wrap(userDto.getPassword()
)));

        User savedUser = userRepository.save(user);
        return userMapper.toUserDto(savedUser);
    }

    // Получение пользователя по логину
    public User findByLogin(String login) {
        return userRepository.getByLogin(login);
    }

    // Получение пользователя по идентификатору
    public User getUserById(Long id) {
```

```

        return userRepository.findById(id);
    }
}
/*
Назначение: сервисный класс для управления участниками мероприятий –
добавление, удаление, поиск
Входные данные: идентификаторы мероприятий и пользователей, DTO
участников
Результат: добавление участника в мероприятие, удаление, получение
участника
ФИО: Муромцев Сергей Андреевич
Дата написания: 30.05.2025 г.
Версия: 1.0
*/
@Service
@RequiredArgsConstructor
@Slf4j
public class ParticipantsService {
    @Lazy
    @Autowired
    private EventService eventService;

    private final UserService userService;
    private final ParticipantMapper participantMapper;
    private final ParticipantRepository participantRepository;

    // Добавление участника к мероприятию
    @Transactional
    public ParticipantDto addParticipantToEvent(ParticipantDto
participantDto) {
        User user = userService.findById(login(participantDto.getLogin()));
        Event event =
eventService.findById(participantDto.getEventId());
        log.info("Adding participant to event {}", event);

        Participant participant =
participantMapper.toParticipant(participantDto);
        participant.setRole(UserEventRole.PARTICIPANT);
        participant.setUser(user);
        participant.setEvent(event);

        log.info("Participant {} added to event",
participantMapper.toParticipantDto(participant));
        return
participantMapper.toParticipantDto(participantRepository.save(participant));
    }

    // Удаление участника из мероприятия
    @Transactional
    public void deleteParticipantFromEvent(Long eventId, String login) {
        log.info("Deleting participant {} from event {}", login,
eventId);

        Participant participant =
participantRepository.findByIdAndUserLogin(eventId, login);

        participantRepository.delete(participant);
        log.info("Participant {} deleted from event {}", login,
eventId);
    }

    // Поиск участника по ID пользователя и ID мероприятия
    public Participant findParticipantByUserIdAndEventId(Long userId,
Long eventId) {

```

```

        return participantRepository.findByIdAndUserId(eventId,
userId);
    }
}
/*
Назначение: сервисный класс для управления мероприятиями — создание,
обновление, удаление, получение по различным критериям
Входные данные: идентификаторы пользователей и мероприятий, DTO событий
Результат: создание и изменение мероприятий, возврат информации о
мероприятиях
ФИО: Муромцев Сергей Андреевич
Дата написания: 30.05.2025 г.
Версия: 1.0
*/
@Slf4j
@Service
@RequiredArgsConstructor
public class EventService {
    private final EventRepository eventRepository;
    private final EventMapper eventMapper;
    private final UserService userService;
    @Lazy
    private final ParticipantsService participantsService;

    // Получение мероприятия по ID
    public Event findById(Long id) {
        return eventRepository.findById(id);
    }

    // Создание нового мероприятия и назначение организатора
    public EventDto createEvent(Long userId, EventDto eventDto) {
        User user = userService.getUserById(userId);
        log.info("Creating new event by {}", user.getLogin());

        Event event = eventMapper.toEvent(eventDto);
        Event savedEvent = eventRepository.save(event);

        Participant participant = new Participant();
        participant.setUser(user);
        participant.setEvent(savedEvent);
        participant.setRole(UserEventRole.ORGANIZER);
        savedEvent.getParticipants().add(participant);

        log.info("Creating new event {}", savedEvent.getTitle());
        return eventMapper.toEventDto(eventRepository.save(savedEvent));
    }

    // Обновление существующего мероприятия
    @Transactional
    public EventDto updateEvent(EventDto eventDto, long eventId) {
        Event existingEvent = eventRepository.findById(eventId);
        log.info("Updating event: {} (id={})", existingEvent.getTitle(),
eventId);

        eventMapper.updateEventFromDto(eventDto, existingEvent);
        Event updatedEvent = eventRepository.update(existingEvent);

        log.info("Event {} updated successfully", updatedEvent.getId());
        return eventMapper.toEventDto(updatedEvent);
    }

    // Удаление мероприятия по ID
    @Transactional

```

```

        public void deleteEvent(Long eventId) {
            Event event = eventRepository.findById(eventId);
            log.info("Delete event: {} (id={})", event.getTitle(), eventId);
            eventRepository.delete(event);
        }

        // Получение мероприятия по ID пользователя
        public EventDto getEventById(Long userId) {
            Event event = eventRepository.findById(userId);
            return eventMapper.toEventDto(event);
        }

        // Получение мероприятия с участником по ID пользователя и
        мероприятия
        public EventDto getEventByIdAndUserId(Long userId, Long eventId) {
            Event event = eventRepository.findById(userId);
            Participant participant =
participantsService.findParticipantByUserIdAndEventId(eventId, userId);
            event.setParticipants(List.of(participant));
            return eventMapper.toEventDto(event);
        }

        // Получение всех мероприятий пользователя с его ролями
        public List<EventDto> getEventsByUserIdWithRoles(Long userId) {
            List<Object[]> results =
eventRepository.findEventsWithRolesByUserId(userId);

            return results.stream()
                .map(result -> {
                    Event event = (Event) result[0];
                    UserEventRole role = (UserEventRole) result[1];

                    EventDto dto = eventMapper.toEventDto(event);
                    dto.setUserEventRole(role);
                    return dto;
                })
                .collect(Collectors.toList());
        }

        // Получение всех мероприятий пользователя
        public List<EventDto> getEventsByUserId(Long userId) {
            List<Event> events = eventRepository.findById(userId);
            return eventMapper.toEventDtos(events);
        }
    }

```

Контроллеры:

```

/*
    Назначение: REST-контроллер для управления пользователями — получение и
    обновление информации о пользователе
    Входные данные: данные аутентификации (@AuthenticationPrincipal), DTO
    пользователя (UserDto)
    Результат: возврат информации о пользователе, обновление данных
    пользователя
    ФИО: Муромцев Сергей Андреевич
    Дата написания: 03.06.2025 г.
    Версия: 1.0
    */
@RestController
@RequestMapping("/user")
@RequiredArgsConstructor
public class UserController {

```

```

        @GetMapping
        public ResponseEntity<?> getUserInfo(@AuthenticationPrincipal
        UserDto userDto) {
            if (userDto == null) {
                return
                ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("User is not
                authenticated");
            }
            return ResponseEntity.ok(userDto);
        }

        @PutMapping
        public ResponseEntity<?> updateUserInfo(@AuthenticationPrincipal
        UserDto userDto, @RequestBody UserDto updatedUserDto) {
            if (userDto == null) {
                return
                ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("User is not
                authenticated");
            }
            userDto.setFirstName(updatedUserDto.getFirstName());
            userDto.setLastName(updatedUserDto.getLastName());
            // Логика обновления пользователя в базе должна быть здесь

            return ResponseEntity.ok(userDto);
        }
    }

    /*
    Назначение: REST-контроллер для управления участниками мероприятий –
    добавление и удаление участников
    Входные данные: данные аутентификации (@AuthenticationPrincipal), DTO
    участников (ParticipantDto), параметры пути
    Результат: добавление участника к мероприятию, удаление участника
    ФИО: Муромцев Сергей Андреевич
    Дата написания: 03.06.2025 г.
    Версия: 1.0
    */
    @RestController
    @RequestMapping("/main/participants")
    @RequiredArgsConstructor
    public class ParticipantsController {

        private final ParticipantsService participantService;

        @PostMapping
        public ResponseEntity<?>
        addParticipantToEvent(@AuthenticationPrincipal UserDto userDto,
                                @RequestBody @Valid
                                ParticipantDto participantDto) {
            if (userDto == null) {
                return
                ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("User not
                authenticated");
            }

            ParticipantDto addedParticipantDto =
            participantService.addParticipantToEvent(participantDto);
            return
            ResponseEntity.status(HttpStatus.CREATED).body(addedParticipantDto);
        }

        @DeleteMapping("/{eventId}/{login}")

```



```

        public ResponseEntity<?> deleteParticipantFromEvent(@PathVariable
Long eventId,
                                                                    @PathVariable
String login) {
        participantService.deleteParticipantFromEvent(eventId, login);
        return ResponseEntity.noContent().build();
    }
}

/*
    Назначение: REST-контроллер для управления мероприятиями – создание,
    обновление, удаление и получение мероприятий
    Входные данные: данные аутентификации (@AuthenticationPrincipal), DTO
    мероприятий (EventDto), параметры пути
    Результат: управление мероприятиями с ответами HTTP
    ФИО: Муромцев Сергей Андреевич
    Дата написания: 03.06.2025 г.
    Версия: 1.0
    */
@RestController
@RequestMapping("/main/events")
@RequiredArgsConstructor
public class EventController {

    private final EventService eventService;

    @PostMapping
    public ResponseEntity<?> createEvent(@AuthenticationPrincipal
UserDto userDto,
                                                                    @RequestBody @Valid EventDto
eventDto) {
        if (userDto == null) {
            return
ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("User not
authenticated");
        }

        EventDto createdEvent =
eventService.createEvent(userDto.getId(), eventDto);
        return
ResponseEntity.status(HttpStatus.CREATED).body(createdEvent);
    }

    @PutMapping("/{eventId}")
    public ResponseEntity<?> updateEventById(@AuthenticationPrincipal
UserDto userDto,
                                                                    @RequestBody @Valid
EventDto eventDto,
                                                                    @PathVariable long eventId)
    {
        if (userDto == null) {
            return
ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("User not
authenticated");
        }

        EventDto event = eventService.updateEvent(eventDto, eventId);
        return ResponseEntity.ok(event);
    }

    @DeleteMapping("/{eventId}")
    public ResponseEntity<?> deleteEventById(@AuthenticationPrincipal
UserDto userDto,

```

```

                                                                 @PathVariable long eventId)
{
    if (userDto == null) {
        return
        ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("User not
        authenticated");
    }

    eventService.deleteEvent(eventId);
    return ResponseEntity.noContent().build();
}

@GetMapping
public ResponseEntity<List<EventDto>>
getEventsByUser(@AuthenticationPrincipal UserDto userDto) {
    if (userDto == null) {
        return
        ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Collections.emptyList());
    }

    List<EventDto> events =
    eventService.getEventsByUserIdWithRoles(userDto.getId());
    return ResponseEntity.ok(events);
}

@GetMapping("/{eventId}")
public ResponseEntity<?> getEventById(@AuthenticationPrincipal
UserDto userDto,
                                                                 @PathVariable long eventId) {
    if (userDto == null) {
        return
        ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Collections.emptyList());
    }

    EventDto event = eventService.getEventById(eventId);
    return ResponseEntity.ok(event);
}
}

```

Конфигурации:

```

/*
Назначение: настройка CORS для обеспечения взаимодействия с фронтом
Входные данные: отсутствуют
Результат: зарегистрированный CorsFilter с настройками разрешенных
источников, заголовков и методов
ФИО: Муромцев Сергей Андреевич
Дата написания: 30.05.2025 г.
Версия: 1.1
*/
@Configuration
@EnableWebMvc
public class WebConfig {

    private static final Long MAX_AGE = 3600L;
    private static final int CORS_FILTER_ORDER = -102;

    @Bean
    public FilterRegistrationBean corsFilter() {
        UrlBasedCorsConfigurationSource source = new
        UrlBasedCorsConfigurationSource();
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowCredentials(true);
        config.addAllowedOrigin("http://localhost:3000");
    }
}

```

```

        config.setAllowedHeaders(Arrays.asList(
            HttpHeaders.AUTHORIZATION,
            HttpHeaders.CONTENT_TYPE,
            HttpHeaders.ACCEPT));
        config.setAllowedMethods(Arrays.asList(
            HttpMethod.GET.name(),
            HttpMethod.POST.name(),
            HttpMethod.PUT.name(),
            HttpMethod.DELETE.name()));
        config.setMaxAge(MAX_AGE);
        source.registerCorsConfiguration("/**", config);
        FilterRegistrationBean bean = new FilterRegistrationBean(new
CorsFilter(source));
        bean.setOrder(CORS_FILTER_ORDER);
        return bean;
    }
}
/*
Назначение: конфигурация безопасности Spring Security с JWT-
аутентификацией
Входные данные: HTTP-запросы, настройки безопасности
Результат: защищённое приложение с разрешёнными маршрутами для /login,
/register и Swagger
ФИО: Муромцев Сергей Андреевич
Дата написания: 30.05.2025 г.
Версия: 1.0
*/
@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final UserAuthenticationEntryPoint
userAuthenticationEntryPoint;
    private final UserAuthenticationProvider userAuthenticationProvider;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
throws Exception {
        http

.exceptionHandling().authenticationEntryPoint(userAuthenticationEntryPoint)
                .and()
                .csrf().disable()

.sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS)
                .and()
                .authorizeHttpRequests((requests) -> requests
                    .requestMatchers(HttpMethod.POST, "/login",
"/register").permitAll()
                    .requestMatchers(
                        "/v3/api-docs/**",
                        "/swagger-ui/**",
                        "/swagger-ui.html",
                        "/main/events"
                    ).permitAll()
                    .anyRequest().authenticated()
                )
                .addFilterBefore(new
JwtAuthFilter(userAuthenticationProvider), BasicAuthenticationFilter.class);

        return http.build();
    }
}

```

```

    }/*
    Назначение: конфигурация Apache Kafka для отправки и получения сообщений
    Входные данные: настройки из application.properties (bootstrap-servers,
group-id)
    Результат: настроенные ProducerFactory, ConsumerFactory, KafkaTemplate и
слушатели
    ФИО: Муромцев Сергей Андреевич
    Дата написания: 30.05.2025 г.
    Версия: 1.0
    */
    @Configuration
    @EnableKafka
    public class KafkaConfig {

        @Value(value = "${spring.kafka.bootstrap-servers}")
        private String bootstrapAddress;

        @Value(value = "${spring.kafka.consumer.group-id}")
        private String groupId;

        @Bean
        public ProducerFactory<String, String> producerFactory() {
            Map<String, Object> configProps = new HashMap<>();
            configProps.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
bootstrapAddress);
            configProps.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class);
            configProps.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
StringSerializer.class);
            return new DefaultKafkaProducerFactory<>(configProps);
        }

        @Bean
        public ConsumerFactory<String, String> consumerFactory() {
            Map<String, Object> props = new HashMap<>();
            props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
bootstrapAddress);
            props.put(ConsumerConfig.GROUP_ID_CONFIG, groupId);
            props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class);
            props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class);
            return new DefaultKafkaConsumerFactory<>(props);
        }

        @Bean
        public ConcurrentKafkaListenerContainerFactory<String, String>
kafkaListenerContainerFactory() {
            ConcurrentKafkaListenerContainerFactory<String, String> factory
=
                new ConcurrentKafkaListenerContainerFactory<>();
            factory.setConsumerFactory(consumerFactory());
            return factory;
        }

        @Bean
        public KafkaTemplate<String, String> kafkaTemplate() {
            return new KafkaTemplate<>(producerFactory());
        }
    }
    /*
    Назначение: конфигурация Kafka-топиков (создание и настройка)

```

Входные данные: адрес Kafka-брокера и имя топика из application.properties
 Результат: зарегистрированный KafkaAdmin и созданный топик для уведомлений
 ФИО: Муромцев Сергей Андреевич
 Дата написания: 30.05.2025 г.
 Версия: 1.0
 */
 @Configuration
 public class KafkaTopicConfig {

 @Value(value = "\${spring.kafka.bootstrap-servers}")
 private String bootstrapAddress;

 @Value("\${spring.kafka.topic.notification.name}")
 private String notification;

 @Bean
 public KafkaAdmin kafkaAdmin() {
 Map<String, Object> configs = new HashMap<>();
 configs.put(AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapAddress);
 return new KafkaAdmin(configs);
 }

 @Bean
 public NewTopic UserInvitedNotificationTopic() {
 return new NewTopic(notification, 1, (short) 1);
 }
 }
}

Классы для работы с уведомлениями:

/*
 Назначение: продюсер для отправки уведомлений о событиях через Kafka
 Входные данные: EventNotification, настроенный KafkaTemplate и
 ObjectMapper
 Результат: сообщение отправляется в указанный Kafka-топик
 ФИО: Муромцев Сергей Андреевич
 Дата написания: 30.05.2025 г.
 Версия: 1.0
 */
 @Component
 public class KafkaEventNotificationsProducer extends
 AbstractProducer<EventNotification> {

 public KafkaEventNotificationsProducer(
 KafkaTemplate<String, String> kafkaTemplate,
 ObjectMapper objectMapper,
 @Value("\${spring.kafka.topic.notification.name}") String
 topicName
) {
 super(kafkaTemplate, topicName, objectMapper);
 }
 }
 /*
 Назначение: слушатель Kafka-сообщений для обработки уведомлений о
 приглашении пользователей
 Входные данные: JSON-сообщения из Kafka-топика
 Результат: десериализованный EventNotification передается в
 NotificationService
 ФИО: Муромцев Сергей Андреевич
 Дата написания: 30.05.2025 г.
 Версия: 1.0

```

*/
@Slf4j
@Component
@RequiredArgsConstructor
public class UserInvitedListener {

    private final NotificationService notificationService;
    private final ObjectMapper objectMapper;

    @KafkaListener(topics = "${spring.kafka.topic.notification.name}",
groupId = "notification-group")
    public void listen(String message) {
        try {
            EventNotification userInvited =
objectMapper.readValue(message, EventNotification.class);
            log.info("Received Kafka message: {}", userInvited);
            notificationService.handleUserInvited(userInvited);
        } catch (JsonProcessingException e) {
            log.error("Failed to parse UserInvited event", e);
        }
    }
}

```

Листинг клиентской логики (React).

```

/*
Назначение: главный компонент фронтенд-приложения управления
мероприятиями,
обеспечивающий переключение тем, навигацию и рендер основных разделов
через маршруты.
Входные данные: состояние темы (темная/светлая), функции переключения
темы и выхода,
маршруты для отображения страниц приложения.
Результат: пользовательский интерфейс с боковой навигацией и основным
контентом,
обновляемым в зависимости от маршрута.
ФИО: Муромцев Сергей Андреевич
Дата написания: 03.06.2025 г.
Версия: 1.0
*/
export default function EventApp() {
    const [isDarkMode, setIsDarkMode] = useState(false);
    const navigate = useNavigate();

    const toggleTheme = () => {
        setIsDarkMode(!isDarkMode);
    };

    const handleLogout = () => {
        setAuthHeader(null);
        localStorage.removeItem("auth_token");
        navigate("/");
        window.location.reload();
    };

    return (
        <div className={` ${styles.appContainer} ${isDarkMode ?
styles.darkMode : styles.lightMode}`}>
            <Sidebar
                isDarkMode={isDarkMode}
                toggleTheme={toggleTheme}
                handleLogout={handleLogout}
            />

```

```

        <div className={styles.mainContent}>
          <Routes>
            <Route path="/" element={<WelcomeEvents />} />
            <Route path="/events" element={<Events isDarkMode={isDarkMode}
/>} />
            <Route path="/calendar" element={<Calendar
isDarkMode={isDarkMode} />} />
            <Route path="/notifications" element={<Notifications
isDarkMode={isDarkMode} />} />
            <Route path="/create-event" element={<CreateEvent
isDarkMode={isDarkMode} />} />
            <Route path="/events/:id/edit" element={<EditEvent
isDarkMode={isDarkMode} />} />
            <Route path="/events/:id/view" element={<ViewEvent
isDarkMode={isDarkMode} />} />
          </Routes>
        </div>
      </div>
    );
  }
  /*

```

Назначение: React-компонент для создания нового мероприятия с заполнением формы и отправкой данных на сервер.

Входные данные: свойства компонента (isDarkMode для темы), локальное состояние формы (название, описание, дата, место, категория), авторизационный токен из localStorage.

Результат: создание мероприятия через POST-запрос, вывод сообщений об успехе или ошибках, переключение состояния загрузки.

ФИО: Муромцев Сергей Андреевич

Дата написания: 03.06.2025 г.

Версия: 1.0

*/

```

export default function CreateEvent({ isDarkMode }) {
  const [title, setTitle] = useState('');
  const [description, setDescription] = useState('');
  const [eventDateTime, setEventDateTime] = useState('');
  const [location, setLocation] = useState('');
  const [eventCategory, setEventCategory] = useState('CONFERENCE');
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);

    const token = localStorage.getItem("auth_token");
    if (!token) {
      alert("Вы не авторизованы");
      setLoading(false);
      return;
    }

    const eventDto = {
      title,
      description,
      eventDateTime,
      location,
      eventCategory
    };

    try {
      const response = await axios.post(
        'http://localhost:8080/main/events',
        eventDto,

```

```

        {
          headers: {
            'Authorization': `Bearer ${token}`,
            'Content-Type': 'application/json'
          },
          withCredentials: true
        }
      );

      alert("Мероприятие создано!");
      console.log(response.data);
    } catch (error) {
      console.error("Ошибка создания мероприятия:", error);
      if (error.response) {
        alert(`Ошибка: ${error.response.data.message} || "Не удалось
создать мероприятие"`);
      } else {
        alert("Не удалось отправить запрос. Проверьте подключение.");
      }
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className={appStyles.mainContent}>
      <h2 className={appStyles.mainTitle}>Создать мероприятие</h2>

      <div className={` ${formStyles.createEventFormWrapper} ${isDarkMode
? appStyles.darkMode : appStyles.lightMode}`}>
        <form onSubmit={handleSubmit}>
          <div className={formStyles.formGroup}>
            <label htmlFor="title">Название мероприятия:</label>
            <input
              id="title"
              type="text"
              value={title}
              onChange={(e) => setTitle(e.target.value)}
              className={formStyles.input}
              placeholder="Введите название"
              required
            />
          </div>

          <div className={formStyles.formGroup}>
            <label htmlFor="description">Описание:</label>
            <textarea
              id="description"
              value={description}
              onChange={(e) => setDescription(e.target.value)}
              className={formStyles.textarea}
              placeholder="Введите описание мероприятия"
              required
            />
          </div>

          <div className={formStyles.formGroup}>
            <label htmlFor="eventDateTime">Дата и время:</label>
            <input
              id="eventDateTime"
              type="datetime-local"
              value={eventDateTime}
              onChange={(e) => setEventDateTime(e.target.value)}
            />
          </div>
        </form>
      </div>
    </div>
  );

```



```

        className={formStyles.input}
        required
      />
    </div>

    <div className={formStyles.formGroup}>
      <label htmlFor="location">Место проведения:</label>
      <input
        id="location"
        type="text"
        value={location}
        onChange={(e) => setLocation(e.target.value)}
        className={` ${formStyles.input}
${formStyles.locationInput}`}
        placeholder="Введите место"
        required
      />
    </div>

    <div className={formStyles.formGroup}>
      <label htmlFor="eventCategory">Категория:</label>
      <select
        id="eventCategory"
        value={eventCategory}
        onChange={(e) => setEventCategory(e.target.value)}
        className={` ${formStyles.select} ${formStyles.input}`}
        required
      >
        <option value="">Выберите категорию</option>
        <option value="CONFERENCE">Конференция</option>
        <option value="WORKSHOP">Мастер-класс</option>
        <option value="MEETUP">Встреча</option>
        <option value="SEMINAR">Семинар</option>
        <option value="WEBINAR">Вебинар</option>
        <option value="HACKATHON">Хакатон</option>
        <option value="TRAINING">Обучение</option>
      </select>
    </div>

    <div className={formStyles.formGroup}>
      <button
        type="submit"
        className={formStyles.submitButton}
        disabled={loading}
      >
        {loading ? "Создание..." : "Создать"}
      </button>
    </div>
  </form>
</div>
</div>
);
}
/*

```

Назначение: React-компонент для отображения списка мероприятий пользователя в виде календаря с возможностью просмотра и редактирования.

Входные данные: isDarkMode – флаг темы, локальное состояние – список событий, статус загрузки и ошибки, токен авторизации из localStorage.

Результат: загрузка и отображение мероприятий с краткой информацией, форматированием даты и возможностью перехода к просмотру и редактированию.

ФИО: Муромцев Сергей Андреевич

Дата написания: 03.06.2025 г.

Версия: 1.0

```

*/
export default function Calendar({ isDarkMode }) {
  const [events, setEvents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  const [filterDate, setFilterDate] = useState("");
  const [filterCategory, setFilterCategory] = useState("");
  const [filterLocation, setFilterLocation] = useState("");

  const truncateText = (text, maxLength = 15) => {
    return text && text.length > maxLength ? text.slice(0, maxLength) +
    "..." : text;
  };

  const EVENT_CATEGORIES = {
    CONFERENCE: "Конференция",
    WORKSHOP: "Мастер-класс",
    MEETUP: "Встреча",
    SEMINAR: "Семинар",
    WEBINAR: "Вебинар",
    HACKATHON: "Хакатон",
    TRAINING: "Обучение"
  };

  const getCategoryKeys = () => Object.keys(EVENT_CATEGORIES);

  useEffect(() => {
    const fetchEvents = async () => {
      try {
        const token = localStorage.getItem("auth_token");
        if (!token) {
          setError("Вы не авторизованы");
          setLoading(false);
          return;
        }

        const response = await
        axios.get("http://localhost:8080/main/events", {
          headers: {
            Authorization: `Bearer ${token}`,
          },
          withCredentials: true,
        });

        setEvents(response.data);
        setLoading(false);
      } catch (err) {
        console.error("Ошибка загрузки мероприятий:", err);
        setLoading(false);
      }
    };

    fetchEvents();
  }, []);

  if (loading) {
    return <div>Загрузка...</div>;
  }

  if (error) {
    return <div>{error}</div>;
  }
}

```

```

const formatDate = (isoString) => {
  const date = new Date(isoString);
  return new Intl.DateTimeFormat('ru-RU', {
    day: 'numeric',
    month: 'long',
    year: 'numeric',
    hour: '2-digit',
    minute: '2-digit'
  }).format(date);
};

const parseDate = (isoString) => new
Date(isoString).toISOString().split("T")[0];

const filteredEvents = events
  .filter(event => {
    const eventDate = parseDate(event.eventDateTime);
    const selectedDate = filterDate;

    if (selectedDate && eventDate !== selectedDate) return false;

    if (filterCategory && event.eventCategory !== filterCategory)
return false;

    if (
      filterLocation &&
!event.location.toLowerCase().includes(filterLocation.toLowerCase())
    )
      return false;

    return true;
  })
  .sort((a, b) => {
    return a.location.localeCompare(b.location);
  });

return (
  <div className={appStyles.mainContent}>
    <h2 className={appStyles.mainTitle}>Календарь</h2>

    <div className={appStyles.filters}>
      <div>
        <label>
          Фильтр по дате:
          <input
            type="date"
            value={filterDate}
            onChange={(e) => setFilterDate(e.target.value)}
          />
        </label>
      </div>

      <div>
        <label>
          Фильтр по категории:
          <select
            value={filterCategory}
            onChange={(e) => setFilterCategory(e.target.value)}
          >
            <option value="">Все катерории</option>
            {getCategoryKeys().map((key) => (

```

```

        <option key={key} value={key}>
          {EVENT_CATEGORIES[key]}
        </option>
      )}}
    </select>
  </label>
</div>

<div>
  <label>
    Фильтр по месту:
    <input
      type="text"
      placeholder="Введите место..."
      value={filterLocation}
      onChange={(e) => setFilterLocation(e.target.value)}
    />
  </label>
</div>
</div>

<div className={` ${eventStyles.eventListContainer} ${isDarkMode ?
appStyles.darkMode : appStyles.lightMode}`}>
  {filteredEvents.length === 0 ? (
    <p>Нет мероприятий по заданным критериям.</p>
  ) : (
    filteredEvents.map((event) => (
      <div
        key={event.id}
        className={` ${eventStyles.eventCard} ${
          event.userEventRole === 'ORGANIZER'
            ? eventStyles.organizerCard
            : event.userEventRole === 'PARTICIPANT'
            ? eventStyles.participantCard
            : ''
        }`}
      >
        <Link
          to={` /main/events/${event.id}/view`}
          className={eventStyles.eventCardLink}
        >
          <div className={eventStyles.cardContent}>
            <h3>{truncateText(event.title, 23)}</h3>

            {event.eventDateTime && (
              <p>
                <strong>Дата:</strong>
                {formatDate(event.eventDateTime)}
              </p>
            )}

            <p><strong>Место:</strong>
            {truncateText(event.location, 20)}</p>
            <p><strong>Описание:</strong>
            {truncateText(event.description, 15)}</p>
            <p>
              <strong>Категория:</strong>{" "}
              {event.eventCategory
                ? EVENT_CATEGORIES[event.eventCategory] ||
event.eventCategory
                : "Не указано"}
            </p>
          </div>
        </Link>
      </div>
    )
  )}

```

```

        Вы:{" "}
        {event.userEventRole === 'ORGANIZER'
        ? 'Организатор'
        : event.userEventRole === 'PARTICIPANT'
        ? 'Участник'
        : 'Неизвестно'}
    </p>
</div>
</Link>

{event.userEventRole === 'ORGANIZER' && (
    <Link
        to={` /main/events/${event.id}/edit`}
        className={eventStyles.editButtonInsideCard}
        onClick={(e) => e.stopPropagation()}
    >
        <button
            className={eventStyles.editButton}
            type="button"
        >
            Редактировать
        </button>
    </Link>
    )}
</div>
))
)}
</div>
</div>
);
}

```

ПРИЛОЖЕНИЕ Г

Тесты программы

```
/*
Тесты для сервиса UserService
Проверяется логика логина, регистрации и получения пользователя
Автор: Муромцев С.А.
*/
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @InjectMocks
    private UserService userService;

    @Mock
    private UserRepository userRepository;

    @Mock
    private PasswordEncoder passwordEncoder;

    @Mock
    private UserMapper userMapper;

    private final String rawPassword = "password123";
    private final String encodedPassword = "$2a$10$encodedPassword";

    private User user;
    private CredentialsDto credentialsDto;
    private SignUpDto signUpDto;
    private UserDto userDto;

    @BeforeEach
    void setUp() {
        user = new User();
        user.setId(1L);
        user.setLogin("testUser");
        user.setPassword(encodedPassword);

        credentialsDto = new CredentialsDto();
        credentialsDto.setLogin("testUser");
        credentialsDto.setPassword(rawPassword.toCharArray());

        signUpDto = new SignUpDto();
        signUpDto.setLogin("newUser");
        signUpDto.setPassword(rawPassword.toCharArray());

        userDto = new UserDto();
        userDto.setId(1L);
        userDto.setLogin("testUser");
    }

    @Test
    void testLoginSuccessful() {
        // Успешный логин
        when(userRepository.getByLogin("testUser")).thenReturn(user);
        when(passwordEncoder.matches(CharBuffer.wrap(rawPassword),
encodedPassword)).thenReturn(true);
        when(userMapper.toUserDto(user)).thenReturn(userDto);

        UserDto result = userService.login(credentialsDto);

        assertEquals(userDto.getLogin(), result.getLogin());
    }
}
```

```

@Test
void testLoginInvalidPassword() {
    // Логин с неверным паролем
    when(userRepository.getByLogin("testUser")).thenReturn(user);
    when(passwordEncoder.matches(CharBuffer.wrap(rawPassword),
encodedPassword)).thenReturn(false);

    AppException ex = assertThrows(AppException.class, () ->
userService.login(credentialsDto));
    assertEquals("Invalid password", ex.getMessage());
}

@Test
void testRegisterSuccessful() {
    // Успешная регистрация

    when(userRepository.findByLogin("newUser")).thenReturn(Optional.empty());
    when(userMapper.signUpToUser(signUpDto)).thenReturn(user);

    when(passwordEncoder.encode(CharBuffer.wrap(rawPassword))).thenReturn(encoded
Password);

    when(userRepository.save(any(User.class))).thenReturn(user);
    when(userMapper.toUserDto(user)).thenReturn(userDto);

    UserDto result = userService.register(signUpDto);

    assertEquals(userDto.getLogin(), result.getLogin());
    verify(userRepository).save(any(User.class));
}

@Test
void testRegisterLoginAlreadyExists() {
    // Попытка регистрации с занятым логином

    when(userRepository.findByLogin("newUser")).thenReturn(Optional.of(user));

    AppException ex = assertThrows(AppException.class, () ->
userService.register(signUpDto));
    assertEquals("Login already exists", ex.getMessage());
}

@Test
void testFindByLogin() {
    // Поиск пользователя по логину
    when(userRepository.getByLogin("testUser")).thenReturn(user);

    User result = userService.findByLogin("testUser");

    assertEquals(user.getLogin(), result.getLogin());
}

@Test
void testGetUserById() {
    // Получение пользователя по ID
    when(userRepository.findById(1L)).thenReturn(user);

    User result = userService.getUserById(1L);

    assertEquals(user.getId(), result.getId());
}
}
/*

```

```

Тесты для ParticipantsService
Проверяется добавление, удаление и поиск участника события
Автор: Муромцев С.А.
*/
class ParticipantsServiceTest {

    @InjectMocks
    private ParticipantsService participantsService;

    @Mock
    private UserService userService;

    @Mock
    private ParticipantMapper participantMapper;

    @Mock
    private ParticipantRepository participantRepository;

    @Mock
    private EventService eventService;

    private final String login = "user1";
    private final Long eventId = 100L;

    @BeforeEach
    void setUp() throws Exception {
        MockitoAnnotations.openMocks(this);

        // Установка @Lazy зависимости через reflection
        Field field =
ParticipantsService.class.getDeclaredField("eventService");
        field.setAccessible(true);
        field.set(participantsService, eventService);
    }

    @Test
    void testAddParticipantToEvent_success() {
        // Успешное добавление участника в событие
        ParticipantDto dto = new ParticipantDto();
        dto.setLogin(login);
        dto.setEventId(eventId);

        User user = new User();
        user.setLogin(login);

        Event event = new Event();
        event.setId(eventId);

        Participant participant = new Participant();
        Participant savedParticipant = new Participant();
        ParticipantDto expectedDto = new ParticipantDto();

        when(userService.findByLogin(login)).thenReturn(user);
        when(eventService.findById(eventId)).thenReturn(event);

        when(participantMapper.toParticipant(dto)).thenReturn(participant);

        when(participantRepository.save(participant)).thenReturn(savedParticipant);

        when(participantMapper.toParticipantDto(savedParticipant)).thenReturn(expected
dDto);
    }
}

```



```

        ParticipantDto result =
participantsService.addParticipantToEvent(dto);

        assertEquals(expectedDto, result);
        assertEquals(UserEventRole.PARTICIPANT, participant.getRole());
        assertEquals(user, participant.getUser());
        assertEquals(event, participant.getEvent());
        verify(participantRepository).save(participant);
    }

    @Test
    void testDeleteParticipantFromEvent_success() {
        // Успешное удаление участника из события
        Participant participant = new Participant();
        when(participantRepository.findByEventIdAndUserLogin(eventId,
login)).thenReturn(participant);

        participantsService.deleteParticipantFromEvent(eventId, login);

        verify(participantRepository).delete(participant);
    }

    @Test
    void testFindParticipantByUserIdAndEventId_success() {
        // Поиск участника по userId и eventId
        Long userId = 10L;
        Participant participant = new Participant();

        when(participantRepository.findByEventIdAndUserId(eventId,
userId)).thenReturn(participant);

        Participant result =
participantsService.findParticipantByUserIdAndEventId(userId, eventId);

        assertEquals(participant, result);
    }
}
/*
Тесты для EventService
Проверяется создание, обновление, удаление и получение событий
Автор: Муромцев С.А.
*/
class EventServiceTest {

    @InjectMocks
    private EventService eventService;

    @Mock
    private EventRepository eventRepository;

    @Mock
    private EventMapper eventMapper;

    @Mock
    private UserService userService;

    @Mock
    private ParticipantsService participantsService;

    private final Long userId = 1L;
    private final Long eventId = 100L;

    @BeforeEach

```

```

void setUp() throws Exception {
    MockitoAnnotations.openMocks(this);

    // @Lazy-зависимость подключается вручную
    Field field =
EventService.class.getDeclaredField("participantsService");
    field.setAccessible(true);
    field.set(eventService, participantsService);
}

@Test
void testCreateEvent_success() {
    // Успешное создание события
    EventDto dto = new EventDto();
    Event event = new Event();
    Event savedEvent = new Event();
    savedEvent.setTitle("Test Event");

    User user = new User();
    user.setLogin("user1");

    when(userService.getUserById(userId)).thenReturn(user);
    when(eventMapper.toEvent(dto)).thenReturn(event);
    when(eventRepository.save(event)).thenReturn(savedEvent);
    when(eventRepository.save(savedEvent)).thenReturn(savedEvent);
    when(eventMapper.toEventDto(savedEvent)).thenReturn(dto);

    EventDto result = eventService.createEvent(userId, dto);

    assertEquals(dto, result);
    verify(eventRepository, times(2)).save(any(Event.class));
}

@Test
void testUpdateEvent_success() {
    // Успешное обновление события
    EventDto dto = new EventDto();
    Event event = new Event();
    Event updatedEvent = new Event();

    when(eventRepository.findById(eventId)).thenReturn(event);
    doNothing().when(eventMapper).updateEventFromDto(dto, event);
    when(eventRepository.update(event)).thenReturn(updatedEvent);
    when(eventMapper.toEventDto(updatedEvent)).thenReturn(dto);

    EventDto result = eventService.updateEvent(dto, eventId);

    assertEquals(dto, result);
    verify(eventRepository).update(event);
}

@Test
void testDeleteEvent_success() {
    // Успешное удаление события
    Event event = new Event();
    event.setId(eventId);
    when(eventRepository.findById(eventId)).thenReturn(event);

    eventService.deleteEvent(eventId);

    verify(eventRepository).delete(event);
}

```

```

@Test
void testGetEventById_success() {
    // Получение события по id
    Event event = new Event();
    EventDto dto = new EventDto();

    when(eventRepository.findById(userId)).thenReturn(event);
    when(eventMapper.toEventDto(event)).thenReturn(dto);

    EventDto result = eventService.getEventById(userId);

    assertEquals(dto, result);
}

@Test
void testGetEventByIdAndUserId_success() {
    // Получение события по id и userId
    Event event = new Event();
    Participant participant = new Participant();
    EventDto dto = new EventDto();

    when(eventRepository.findById(userId)).thenReturn(event);

when(participantsService.findParticipantByUserIdAndEventId(eventId,
userId)).thenReturn(participant);
    when(eventMapper.toEventDto(event)).thenReturn(dto);

    EventDto result = eventService.getEventByIdAndUserId(userId,
eventId);

    assertEquals(dto, result);
    assertEquals(1, event.getParticipants().size());
}

@Test
void testGetEventsByUserIdWithRoles_success() {
    // Получение списка событий с ролями по userId
    Event event = new Event();
    EventDto dto = new EventDto();
    UserEventRole role = UserEventRole.PARTICIPANT;

    when(eventRepository.findEventsWithRolesByUserId(userId))
        .thenReturn(List.of(new Object[]{event, role}));
    when(eventMapper.toEventDto(event)).thenReturn(dto);

    List<EventDto> result =
eventService.getEventsByUserIdWithRoles(userId);

    assertEquals(1, result.size());
    assertEquals(role, result.get(0).getUserEventRole());
}

@Test
void testGetEventsByUserId_success() {
    // Получение всех событий по userId
    Event event = new Event();
    EventDto dto = new EventDto();

when(eventRepository.findByUserId(userId)).thenReturn(List.of(event));

when(eventMapper.toEventDtos(List.of(event))).thenReturn(List.of(dto));

```

```

        List<EventDto> result = eventService.getEventsByUserId(userId);

        assertEquals(1, result.size());
    }
}
/*
Тесты для Kafka
Проверяется корректность отправки, получение и сохранение в БД
уведомления
Автор: Муромцев С.А.
*/
@DirtiesContext
@SpringBootTest
@EmbeddedKafka(
    partitions = 1,
    topics = {"notification"},
    brokerProperties = {
        "listeners=PLAINTEXT://localhost:9092",
        "port=9092"
    }
)
public class KafkaIntegrationTest {

    @Autowired
    private EmbeddedKafkaBroker embeddedKafkaBroker;

    @Autowired
    private KafkaEventNotificationsProducer kafkaProducer;

    @Autowired
    private NotificationService notificationService;

    @Autowired
    private ParticipantService participantService;

    @Autowired
    private UserService userService;

    private final ObjectMapper objectMapper = new ObjectMapper();

    @AfterEach
    void tearDown() {
        try {
            Thread.sleep(1000); // Даём Windows время закрыть дескрипторы
        } catch (InterruptedException ignored) {}
    }

    @Test
    void testEventNotification_isProcessedViaKafka() throws Exception {
        String topic = "notification";
        Long userId = 1L;
        String title = "Новое событие";
        NotificationType type = NotificationType.INVITE;
        String message = "Вы были приглашены!";

        EventNotification eventNotification = new EventNotification(userId,
            title, type, message);

        kafkaProducer.sendMessage(eventNotification);
        System.out.println("Уведомление отправилось");

        Map<String, Object> consumerProps =
            KafkaTestUtils.consumerProps("testGroup", "true", embeddedKafkaBroker);

```

```

        DefaultKafkaConsumerFactory<String, String> consumerFactory = new
DefaultKafkaConsumerFactory<>(consumerProps, new StringDeserializer(), new
StringDeserializer());
        Consumer<String, String> consumer = consumerFactory.createConsumer();
        consumer.subscribe(Collections.singletonList(topic));

        ConsumerRecords<String, String> records =
consumer.poll(Duration.ofSeconds(5));
        assertThat(records).isNotEmpty();

        Optional<ConsumerRecord<String, String>> found =
StreamSupport.stream(records.splititerator(), false)
                .filter(record -> {
                    try {
                        EventNotification received =
objectMapper.readValue(record.value(), EventNotification.class);
                        return received.userId().equals(userId);
                    } catch (Exception e) {
                        return false;
                    }
                })
                .findFirst();

        assertThat(found).isPresent();
        System.out.println("Уведомление обработалось");
        consumer.close();
    }
}

```